

COMPASS_Synop- tic_LE_Tree_CO2_Flux_Calcs_2022

Roberta Peixoto

2026-04-24

Table of contents

1	Add Required Packages	2
2	Pull in Raw Datafiles	2
3	Read in metadata file and match to raw data	7
4	Check start times overlaps or missing values	8
5	Match metadata to raw data	10
6	Reading matched data and metadata	14
7	Temperature from weather station in the wetland	14
8	fixing some Deadbands	18
9	Calculate fluxes	18
10	check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)	24
11	Spot check a few fluxes with $R^2 > 0.9$	25
12	Take a look at the fluxes < 0.9 and a flux estimate below 1	30
13	Take a look at the bad fluxes < 0.9 and have a high flux estimate	33
14	See if we want to remove these fluxes or if we want to try and recalculate these ones?	33

15 Look at the really high or really low ones to see if there is a timing issue or something	37
16 Check all fluxes	39
17 Overall checks fo r2	42
18 Remove any fluxes that do not fit criteria	43
19 Do some visualization of QAQC plots to assess fluxes	44
20 Read out final dataframe and export for use elsewhere	47

1 Add Required Packages

2 Pull in Raw Datafiles

```
#defining where the raw data is

DATA_DIR_ROOT <- here::here(
  "Tree_Fluxes_Synoptic_LE",
  "2022",
  "Raw Data"
)

#fucntion to read data
now <- function() format(Sys.time(), "%Y-%m-%d %H:%M:%S")
message(now(), " | Script started")
```

2026-04-24 15:24:50 | Script started

```
#finding the zip files
message(now(), " | Looking for ZIP files in: ", DATA_DIR_ROOT)
```

2026-04-24 15:24:50 | Looking for ZIP files in: D:/GitHub/LE-Synoptic-discrete-data/Tree_Fluxes_Synoptic_LE/2022/Raw Data

```
zip_files <- list.files(
  DATA_DIR_ROOT,
  pattern = "\\\\.zip$",
  full.names = TRUE)

message(now(), " | ZIP files found: ", length(zip_files))
```

2026-04-24 15:24:50 | ZIP files found: 1

```
print(zip_files)
```

```
[1] "D:/GitHub/LE-Synoptic-discrete-data/Tree_Fluxes_Synoptic_LE/2022/Raw Data/TG10-01028-TG10-01448-TG10-01426.zip"
```

```
if (length(zip_files) == 0) {
  stop("No ZIP files found. Check DATA_DIR_ROOT.")}

cache <- list()

# Reader function
read_li7810_zip <- function(zip_file, tz = "US/Eastern") {
  message(now(), " | Processing ZIP: ", basename(zip_file))
  if (zip_file %in% names(cache)) {
    message(" -> Using cached version")
    return(cache[[zip_file]]) }

  tmp_dir <- tempfile("li7810_")
  dir.create(tmp_dir)
  on.exit(unlink(tmp_dir, recursive = TRUE), add = TRUE)
  unzip(zip_file, exdir = tmp_dir)
  data_files <- list.files(
    tmp_dir,
    pattern = "\\\\.data$",
    full.names = TRUE,
    recursive = TRUE )

  message(" -> .data files found: ", length(data_files))
  if (length(data_files) == 0) {
    stop("No .data files inside ZIP: ", zip_file) }
```

```

dat_list <- lapply(data_files, function(f) {
  message("      Reading: ", basename(f))
  ffi_read_LI7810(f)})

dat <- data.table::rbindlist(dat_list, fill = TRUE) %>%
  mutate(TIMESTAMP = with_tz(TIMESTAMP, tz))
cache[[zip_file]] <- dat
message(" -> ZIP finished: ", basename(zip_file))
return(dat)}

# ACTUAL EXECUTION
message(now(), " | Starting data read")

```

2026-04-24 15:24:50 | Starting data read

```
alldat_list <- lapply(zip_files, read_li7810_zip)
```

2026-04-24 15:24:50 | Processing ZIP: TG10-01028-TG10-01448-TG10-01426.zip

-> .data files found: 13

Reading: TG10-01028-2022-07-01T080000.data

TG10-01028-2022-07-01T080000.data: read 479273 rows of TG10-01028 data, 2022-07-05 07:53:57 to 2022-11-08 15:04:55 US/Eastern

Reading: TG10-01028-2022-07-01T120000 (1).data

TG10-01028-2022-07-01T120000 (1).data: read 650394 rows of TG10-01028 data, 2022-07-05 07:53:57 to 2023-05-10 13:16:52 US/Eastern

Reading: TG10-01028-2022-07-01T120000 (2).data

TG10-01028-2022-07-01T120000 (2).data: read 670798 rows of TG10-01028 data, 2022-07-05 07:53:57 to 2023-05-30 15:07:38 US/Eastern

Reading: TG10-01028-2022-07-01T120000.data

TG10-01028-2022-07-01T120000.data: read 261828 rows of TG10-01028 data, 2022-07-05 07:53:57 to 2022-08-25 09:40:11 America/New_York

Reading: TG10-01028-2023-05-31T120000.data

TG10-01028-2023-05-31T120000.data: read 245830 rows of TG10-01028 data, 2023-05-31 14:59:59 to 2023-07-14 08:47:43 US/Eastern

Reading: TG10-01286-2022-07-17T120000.data

TG10-01286-2022-07-17T120000.data: read 77353 rows of TG10-01286 data, 2022-07-19 11:08:05 to 2022-07-21 12:00:00 US/Eastern

Reading: TG10-01426-2022-07-01T080000.data

TG10-01426-2022-07-01T080000.data: read 1556244 rows of TG10-01426 data, 2022-09-16 12:56:33 to 2022-10-19 12:58:47 EST5EDT

Reading: TG10-01426-2023-01-01T080000.data

TG10-01426-2023-01-01T080000.data: read 135486 rows of TG10-01426 data, 2023-03-16 14:47:52 to 2023-09-28 12:12:24 EST5EDT

Reading: TG10-01426-2023-04-01T120000.data

TG10-01426-2023-04-01T120000.data: read 140657 rows of TG10-01426 data, 2023-07-14 15:44:04 to 2023-10-05 10:04:26 EST5EDT

Reading: TG10-01448-2022-07-01T120000.data

TG10-01448-2022-07-01T120000.data: read 2089581 rows of TG10-01448 data, 2022-09-15 15:08:54 to 2023-05-30 13:48:41 EST5EDT

Reading: TG10-01448-2022-12-01T120000.data

TG10-01448-2022-12-01T120000.data: read 221778 rows of TG10-01448 data, 2022-12-05 16:36:14 to 2023-05-30 13:48:41 EST5EDT

Reading: TG10-01448-2023-04-01T120000.data

TG10-01448-2023-04-01T120000.data: read 485093 rows of TG10-01448 data, 2023-04-05 13:35:59 to 2023-10-05 09:57:17 EST5EDT

Reading: TG10-01448-2023-05-01T080000.data

TG10-01448-2023-05-01T080000.data: read 432144 rows of TG10-01448 data, 2023-05-10 10:33:38 to 2023-09-08 12:54:44 EST5EDT

-> ZIP finished: TG10-01028-TG10-01448-TG10-01426.zip

```
alldat <- data.table::rbindlist(alldat_list, fill = TRUE)
message(now(), " | DONE")
```

2026-04-24 15:27:53 | DONE

```
message("Final number of rows: ", nrow(alldat))
```

Final number of rows: 7446459

```
# Show result
print(head(alldat))
```

	DATAH	SECONDS	NANOSECONDS	NDX	DIAG	REMARK	H2O	CO2	CH4	CAVITY_P
	<char>	<int>	<int>	<int>	<int>	<lgcl>	<num>	<num>	<num>	<num>
1:	DATA	1657022037	60688972	28	256	NA	NA	NA	NA	99.1765
2:	DATA	1657022038	60637950	32	256	NA	NA	NA	NA	99.1769
3:	DATA	1657022039	60590028	36	256	NA	NA	NA	NA	99.1774
4:	DATA	1657022040	60548067	40	256	NA	NA	NA	NA	99.1786
5:	DATA	1657022041	60539960	44	256	NA	NA	NA	NA	99.1782
6:	DATA	1657022042	60539960	48	256	NA	NA	NA	NA	99.1794
	CAVITY_T	LASER_PHASE_P	LASER_T	RESIDUAL	RING_DOWN_TIME	THERMAL_ENCLOSURE_T				
	<num>	<num>	<num>	<num>	<num>	<num>				<num>
1:	24.1912	98.5768	28.3186	0		0				24.1020
2:	24.1912	98.5768	38.5134	0		0				24.1022
3:	24.1912	98.5776	38.8638	0		0				24.1022
4:	24.1912	98.5780	38.9849	0		0				24.1022
5:	24.1912	98.5780	38.9638	0		0				24.1025

```

6: 24.1912      98.5788 38.9500      0      0      24.1029
    PHASE_ERROR LASER_T_SHIFT INPUT_VOLTAGE  CHK      TIMESTAMP      TZ
      <num>      <num>      <num> <int>      <POS> <char>
1:   -16000      0      23.9785   37 2022-07-05 07:53:57 US/Eastern
2:   -16000      0      23.9730   16 2022-07-05 07:53:58 US/Eastern
3:   -16000      0      23.9738   79 2022-07-05 07:53:59 US/Eastern
4:   -16000      0      23.9683   13 2022-07-05 07:54:00 US/Eastern
5:   -16000      0      23.9604    9 2022-07-05 07:54:01 US/Eastern
6:   -16000      0      23.9585  234 2022-07-05 07:54:02 US/Eastern

      SN      MODEL
    <char> <char>
1: TG10-01028 LI-7810
2: TG10-01028 LI-7810
3: TG10-01028 LI-7810
4: TG10-01028 LI-7810
5: TG10-01028 LI-7810
6: TG10-01028 LI-7810

```

```

#REMOVING WEIRD VERY LOW VALUES ON "2022-12-20 11:41:00" co2=-4000 licor bug???
alldat<-alldat%>%filter(CO2>-1000)

```

3 Read in metadata file and match to raw data

```

md <- "Raw Data/COMPASS_SynopticLE_TreeFlux_Metadata_2022b.csv"
dat <- read.csv(md)

metadat <- dat %>%
  # Remove rows without date
  filter(!is.na(Date)) %>%
  # Keep only desired instruments
  filter(SN %in% c("TG10-01448", "TG10-01426", "TG10-01028")) %>%
  # Normalize Plot labels
  mutate(
    Plot_raw = toupper(Plot),
    # Extract plot components
    #Site = str_extract(Plot_raw, "[A-Z]{3}"),
    #Zone = str_extract(Plot_raw, "(TR|UP)"), # somehow all of the UP PTR measurements turned
    Site = toupper(Site),
    Zone = toupper(Zone),
    Replicate = str_extract(Plot_raw, "(?<=TR|UP)[0-9]+"),

```

```

# Letter ONLY if it comes AFTER a number
Letter = str_extract(Plot_raw, "(?<=[0-9])[A-Z]$"),
# Time normalization
Time_start = str_replace_all(as.character(Time_start), "\\.", ":") |> str_trim(),
Stop.time = str_replace_all(as.character(Stop.time), "\\.", ":") |> str_trim(),
# Safe time parsing (NO WARNINGS)
t_start = suppressWarnings(parse_date_time(Time_start, orders = c("H:M:S", "H:M"))),
t_stop = suppressWarnings(parse_date_time(Stop.time, orders = c("H:M:S", "H:M"))),
# Observation length
Obs_length = as.numeric(t_stop - t_start, units = "secs"),
# Timestamp
TIMESTAMP = as.POSIXct(
  paste(Date, Time_start),
  format = "%m/%d/%Y %H:%M:%S",
  tz = "EST"
),
# Sequential ID
Seq = row_number(),
# INAL standardized Plot label
Plot = if_else(
  is.na(Letter),
  paste(Date, Site, Zone, Replicate, Seq, sep = "_"),
  paste(Date, Site, Zone, Replicate, Letter, Seq, sep = "_")) )

# Quick check
head(metadata$Plot)

```

```

[1] "12/20/2022_OWC_TR_NA_1" "12/20/2022_OWC_TR_NA_2" "12/20/2022_OWC_TR_NA_3"
[4] "12/20/2022_OWC_TR_NA_4" "12/20/2022_OWC_TR_NA_5" "12/20/2022_OWC_TR_NA_6"

```

4 Check start times overlapsor missing values

```

#checking if there is any start and stop time overlap from the field notes
overlaps_detected <- metadata %>%
  mutate(
    start_ts = as.POSIXct(
      paste(Date, Time_start),
      format = "%m/%d/%Y %H:%M:%S",
      tz = "EST"
    )
  )

```



```

    ),
    end_ts = start_ts + Obs_length
  ) %>%
  arrange(SN, start_ts) %>%
  group_by(SN) %>%
  mutate(
    next_plot      = lead(Plot),
    next_start_ts  = lead(start_ts),
    overlap_fffi   = !is.na(next_start_ts) & next_start_ts <= end_ts
  ) %>%
  filter(overlap_fffi) %>%
  select(
    SN,
    Plot,
    start_ts,
    end_ts,
    next_plot,
    next_start_ts,
    Obs_length
  )

print(overlaps_detected)

```

```

# A tibble: 0 x 7
# Groups:   SN [0]
# i 7 variables: SN <chr>, Plot <chr>, start_ts <dtm>, end_ts <dtm>,
#   next_plot <chr>, next_start_ts <dtm>, Obs_length <dbl>

```

```

#checking problems with obs_length - this is a frequent bug for manual field notes
field_notes_overlap<-metadat %>%
  mutate(
    obs_problem =
      is.na(Obs_length) | Obs_length <= 0
  ) %>%
  filter(obs_problem) %>%
  select(
    SN,
    Date,
    Plot,
    Time_start,
    Stop.time,

```

```

    Obs_length
  )

print(field_notes_overlap)

[1] SN          Date          Plot          Time_start Stop.time  Obs_length
<0 rows> (or 0-length row.names)

```

```

# checking unique measurment start time and stop time
# alldat %>%
#   filter(
#     SN == "TG10-01448",
#     TIMESTAMP > as.POSIXct("2023-01-05 13:28:00"),
#     TIMESTAMP < as.POSIXct("2023-01-05 13:33:00")
#   ) %>%
#   ggplot(aes(x = TIMESTAMP, y = CO2)) +
#   geom_point()#ylim(300,500)

```

5 Match metadata to raw data

```

#function to match data per equipment (considering different serial numbers) uhul!
match_flux_metadata <- function(alldat, metadat) {
  message("Starting metadata Licor matching")
  #Sanity checks (fail early)
  stopifnot(
    "TIMESTAMP" %in% names(alldat),
    "SN" %in% names(alldat),
    "Date" %in% names(metadat),
    "Time_start"%in% names(metadat),
    "Obs_length"%in% names(metadat),
    "Plot" %in% names(metadat)
  )

  stopifnot(all(metadat$Obs_length > 0, na.rm = TRUE))
  stopifnot(anyDuplicated(metadat$Plot) == 0)
  # Instruments present in BOTH datasets
  instruments <- intersect(
    unique(alldat$SN),
    unique(metadat$SN)
  )
}

```

```

)
message("Instruments to process: ", paste(instruments, collapse = ", "))
results <- lapply(instruments, function(sn) {
  message("Processing SN: ", sn)
  dat_i <- alldat %>%
    filter(SN == sn)
  meta_i <- metadat %>%
    filter(SN == sn) %>%
    arrange(Date, Time_start) %>%
    mutate(metadat_row = row_number())
  stopifnot(nrow(meta_i) > 0)
# Actual matching
  dat_i$metadat_row <- ffi_metadata_match(
    data_timestamps = dat_i$TIMESTAMP,
    start_dates      = meta_i$Date,
    start_times      = meta_i$Time_start,
    obs_lengths      = meta_i$Obs_length )
# Attach metadata (vectorized, fast)
  dat_i <- dat_i %>%
    mutate(
      Plot      = meta_i$Plot[metadat_row],
      Site      = meta_i$Site[metadat_row],
      Zone      = meta_i$Zone[metadat_row],
      Volume    = meta_i$Volume[metadat_row],
      Temp      = meta_i$Temp[metadat_row],
      Area      = meta_i$Area[metadat_row],
      Dead_band = meta_i$Dead_band[metadat_row] )
# Drop unmatched rows
  n_before <- nrow(dat_i)
  dat_i <- dat_i %>% filter(!is.na(metadat_row))
  n_after <- nrow(dat_i)
  message("  Matched rows: ", n_after, " / ", n_before)
  dat_i })
alldat_matched <- rbindlist(results, fill = TRUE)
#Final sanity checks
stopifnot(
  !anyNA(alldat_matched$Plot),
  nrow(alldat_matched) > 0
)
message("Matching finished successfully")
return(alldat_matched)}

```

```
#running the function  
message("Running final metadata matching...")
```

Running final metadata matching...

```
alldat_matched <- match_flux_metadata(alldat, metadat)
```

Starting metadata Licor matching

Instruments to process: TG10-01028, TG10-01448

Processing SN: TG10-01028

YYYY-MM-DD format failed for start_dates; trying MM/DD/YYYY

6 entries had no timestamp matches!

Matched rows: 113283 / 2072113

Processing SN: TG10-01448

YYYY-MM-DD format failed for start_dates; trying MM/DD/YYYY

Matched rows: 4080 / 3057529

Matching finished successfully

```
message("Matched dataset rows: ", nrow(alldat_matched))
```

Matched dataset rows: 117363

```
head(alldat_matched)
```

	DATAH	SECONDS	NANOSECONDS	NDX	DIAG	REMARK	H2O	CO2	CH4
	<char>	<int>	<int>	<int>	<int>	<lgcl>	<num>	<num>	<num>
1:	DATA	1660653720	140717029	46468	0	NA	23377.03	437.7346	2226.720
2:	DATA	1660653721	140717029	46472	0	NA	23329.68	438.0153	2226.660
3:	DATA	1660653722	140717029	46476	0	NA	23281.40	438.7122	2226.554
4:	DATA	1660653723	140717029	46480	0	NA	23238.72	438.1288	2226.775
5:	DATA	1660653724	140325069	46484	0	NA	23198.24	438.7191	2226.591
6:	DATA	1660653725	140325069	46488	0	NA	23157.03	438.5513	2226.626

	CAVITY_P	CAVITY_T	LASER_PHASE_P	LASER_T	RESIDUAL	RING_DOWN_TIME
	<num>	<num>	<num>	<num>	<num>	<num>
1:	40.2577	55.001	27.3237	39.4727	0.00345156	12.5877
2:	40.2577	55.001	27.3241	39.4727	0.00345356	12.5883
3:	40.2581	55.001	27.3241	39.4727	0.00346888	12.5880
4:	40.2569	55.001	27.3237	39.4727	0.00345226	12.5885
5:	40.2561	55.001	27.3245	39.4727	0.00345378	12.5886
6:	40.2573	55.001	27.3233	39.4728	0.00345417	12.5887

	THERMAL_ENCLOSURE_T	PHASE_ERROR	LASER_T_SHIFT	INPUT_VOLTAGE	CHK
	<num>	<num>	<num>	<num>	<int>
1:	54.4998	-9.50	0.538616	14.6444	35
2:	54.4998	-2.25	0.538616	14.6444	76
3:	54.4998	-57.25	0.538616	14.6432	86
4:	54.4998	-16.00	0.538616	14.6432	249
5:	54.4998	-17.00	0.538616	14.6440	201
6:	54.4998	-51.75	0.538616	14.6440	84

	TIMESTAMP	TZ	SN	MODEL	metadat_row
	<POSc>	<char>	<char>	<char>	<num>
1:	2022-08-16 08:42:00	US/Eastern	TG10-01028	LI-7810	90
2:	2022-08-16 08:42:01	US/Eastern	TG10-01028	LI-7810	90
3:	2022-08-16 08:42:02	US/Eastern	TG10-01028	LI-7810	90
4:	2022-08-16 08:42:03	US/Eastern	TG10-01028	LI-7810	90
5:	2022-08-16 08:42:04	US/Eastern	TG10-01028	LI-7810	90
6:	2022-08-16 08:42:05	US/Eastern	TG10-01028	LI-7810	90

	Plot	Site	Zone	Volume	Area	Dead_band
	<char>	<char>	<char>	<num>	<num>	<lgcl>
1:	8/16/2022_CRC_TR_316_157	CRC	TR	0.000128704	0.00202683	NA
2:	8/16/2022_CRC_TR_316_157	CRC	TR	0.000128704	0.00202683	NA
3:	8/16/2022_CRC_TR_316_157	CRC	TR	0.000128704	0.00202683	NA
4:	8/16/2022_CRC_TR_316_157	CRC	TR	0.000128704	0.00202683	NA
5:	8/16/2022_CRC_TR_316_157	CRC	TR	0.000128704	0.00202683	NA
6:	8/16/2022_CRC_TR_316_157	CRC	TR	0.000128704	0.00202683	NA

```
#saving matched data and metadata as raw data is too big (today, Licor files leave in the CO2 column)
saveRDS(alldat_matched, file = "alldat_matched_2022.rds")
```

```
#can remove any row that have NA's in the CO2 column
#alldat <- alldat %>%
# filter(!is.na(CO2))
```

#Extracting initial CO2 concentration

```
## Extract initial CO2 concentration (t0) for each Plot for QAQC later
df_CO2_in <- alldat_matched %>%
  arrange(Plot, TIMESTAMP) %>%
  group_by(Plot) %>%
  mutate(
    CO2_initial = first(CO2)
  ) %>% dplyr::select(CO2_initial, Plot) %>%
  ungroup()
```

6 Reading matched data and metadata

```
# after you match the data for the first time - we can just read the .rds file to save time later
alldat_matched <- readRDS("alldat_matched_2022.rds")
```

7 Temperature from weather station in the wetland

```
# Getting Air temp from the weather station
## Read L2 temperature data from ZIP stored outside GitHub,
## extract desired variable, save lightweight CSV,
## and remove temporary unzipped files afterwards

## Read extracted L2 Parquet files and save temperature time series
BASE_DIR <- "D:/COMPASS_L2-v2-1/v2-1"
SITE <- "(OWC|CRC|PTR)"
VARIABLE <- "wx-tempavg15"

OUTFILE <- "Raw Data/L2_temp_timeseries.csv"
```

```
# Build regex that matches REAL L2 filenames
regex <- paste0(SITE, ".*", VARIABLE, ".*_L2_v2-1.*\\.parquet$")

# Locate Parquet files
files <- list.files(BASE_DIR, pattern = regex, recursive = TRUE, full.names = TRUE)

message("Files found: ", length(files))
```

Files found: 12

```
print(basename(files))
```

```
[1] "CRC_W_2022_wx-tempavg15_L2_v2-1.parquet"
[2] "CRC_W_2023_wx-tempavg15_L2_v2-1.parquet"
[3] "CRC_W_2024_wx-tempavg15_L2_v2-1.parquet"
[4] "CRC_W_2025_wx-tempavg15_L2_v2-1.parquet"
[5] "OWC_W_2022_wx-tempavg15_L2_v2-1.parquet"
[6] "OWC_W_2023_wx-tempavg15_L2_v2-1.parquet"
[7] "OWC_W_2024_wx-tempavg15_L2_v2-1.parquet"
[8] "OWC_W_2025_wx-tempavg15_L2_v2-1.parquet"
[9] "PTR_W_2022_wx-tempavg15_L2_v2-1.parquet"
[10] "PTR_W_2023_wx-tempavg15_L2_v2-1.parquet"
[11] "PTR_W_2024_wx-tempavg15_L2_v2-1.parquet"
[12] "PTR_W_2025_wx-tempavg15_L2_v2-1.parquet"
```

```
# Fail fast if nothing is found
stopifnot(length(files) > 0)

# Read and combine Parquet files
dat <- lapply(files, function(f) {
  message("Reading ", basename(f))
  read_parquet(f)
})
```

Reading CRC_W_2022_wx-tempavg15_L2_v2-1.parquet

Reading CRC_W_2023_wx-tempavg15_L2_v2-1.parquet

Reading CRC_W_2024_wx-tempavg15_L2_v2-1.parquet

Reading CRC_W_2025_wx-tempavg15_L2_v2-1.parquet

Reading OWC_W_2022_wx-tempavg15_L2_v2-1.parquet

Reading OWC_W_2023_wx-tempavg15_L2_v2-1.parquet

Reading OWC_W_2024_wx-tempavg15_L2_v2-1.parquet

Reading OWC_W_2025_wx-tempavg15_L2_v2-1.parquet

Reading PTR_W_2022_wx-tempavg15_L2_v2-1.parquet

Reading PTR_W_2023_wx-tempavg15_L2_v2-1.parquet

Reading PTR_W_2024_wx-tempavg15_L2_v2-1.parquet

Reading PTR_W_2025_wx-tempavg15_L2_v2-1.parquet

```
dat <- bind_rows(dat)

# Optional QA filtering (only if column exists)
if ("QA_flag" %in% names(dat)) {
  dat <- dat %>% filter(QA_flag == 0)
}

# Keep only necessary columns
temps <- dat %>%
  select(TIMESTAMP, Plot, Site, Value) %>%
  rename(Temp = Value)

head(temps)
```

```
# A tibble: 6 x 4
  TIMESTAMP          Plot Site   Temp
  <dtm>              <chr> <chr> <dbl>
1 2022-01-01 00:00:00 W     CRC    NA
2 2022-01-01 00:15:00 W     CRC    NA
3 2022-01-01 00:30:00 W     CRC    NA
4 2022-01-01 00:45:00 W     CRC    NA
5 2022-01-01 01:00:00 W     CRC    NA
6 2022-01-01 01:15:00 W     CRC    NA
```



```

# Save lightweight output only
write.csv(
  temps,
  file = OUTFILE,
  row.names = FALSE
)

#reading temp
temps <- read.csv("Raw Data/L2_temp_timeseries.csv")

## Match up the L1 temp data and add it to the metadata dataframe
#Going to need to use site and timestamp in the metadata in order to find a date

#make sure time stamps are in the correct format
alldat_matched$TIMESTAMP <- as.POSIXct(alldat_matched$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS")
temps$TIMESTAMP <- as.POSIXct(temps$TIMESTAMP, format = "%Y-%m-%d %H:%M:%OS")

#time rounding step
alldat_matched$DateTime_round <- round_date(alldat_matched$TIMESTAMP, "60 mins")
temps$TIMESTAMP_round <- round_date(temps$TIMESTAMP, "60 mins")

#ensure there is only one data observation per hour
temps_clean <- temps %>%
  group_by(TIMESTAMP_round, Site) %>%
  summarize(Temp = mean(Temp))

```

`summarise()` has regrouped the output.
 i Summaries were computed grouped by TIMESTAMP_round and Site.
 i Output is grouped by TIMESTAMP_round.
 i Use `summarise(.groups = "drop\_last")` to silence this message.
 i Use `summarise(.by = c(TIMESTAMP\_round, Site))` for per-operation grouping
 (`?dplyr::dplyr\_by`) instead.

```

#join them
alldat_matched <- alldat_matched %>%
  left_join(temps_clean, by=c("Site"="Site", "DateTime_round"="TIMESTAMP_round"))

```

8 fixing some Deadbands

```
# need to include a way to adjust the stop time too
# Table of plots with custom deadband adjustments
deadband_overrides <- tibble::tibble(
  Plot = c(),
  DeadBand_override = c(
  )
)

# Default deadband
alldat_matched <- alldat_matched %>%
  mutate(
    DeadBand_used = as.numeric(Dead_band),
    DeadBand_used = if_else(
      is.na(DeadBand_used),
      0,
      DeadBand_used
    )
  )

# # Apply per-plot overrides
alldat_matched <- alldat_matched %>%
  left_join(deadband_overrides, by = "Plot") %>%
  mutate(
    DeadBand_used = if_else(
      !is.na(DeadBand_override),
      DeadBand_override,
      DeadBand_used
    )
  ) %>%
  select(-DeadBand_override)
```

9 Calculate fluxes

```
# TO Do - NEED TO PULL TEMP from the weather station
#change the units
alldat_matched$CO2_umol <- ffi_ppm_to_umol(alldat_matched$CO2,
```

```

volume = alldat_matched$Volume, # m3
temp = alldat_matched$Temp)      # degrees C from the weather

```

Assuming atm = 101325 Pa

Using R = 8.31446261815324 m3 Pa K-1 mol-1

```

#> Assuming atm = 101325 Pa
#> Using R = 8.31446261815324 m3 Pa K-1 mol-1

alldat_matched$CO2_umol_m2 <- alldat_matched$CO2_umol / 0.16

fluxes <- ffi_compute_fluxes(alldat_matched ,
                             group_column = "Plot",
                             time_column = "TIMESTAMP",
                             gas_column = "CO2_umol_m2",
                             dead_band = 0)

```

NOTE: HM81_flux.estimate is not NA, implying nonlinear data

NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data

[illegible]

[illegible]

NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data
 NOTE: HM81_flux.estimate is not NA, implying nonlinear data

```
head(fluxes)
```

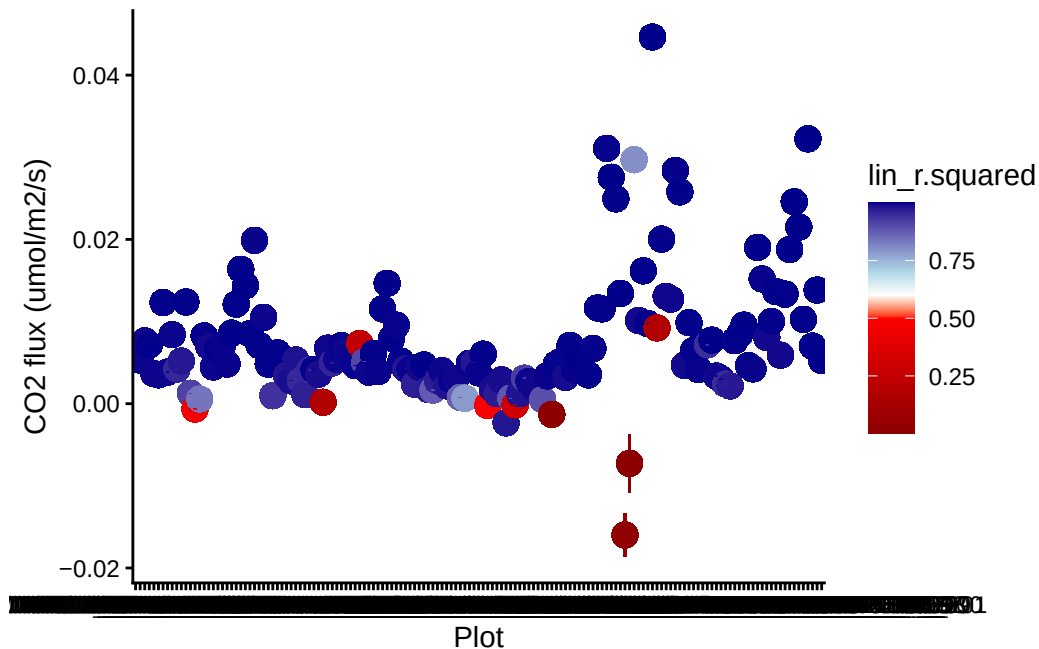
	Plot	TIMESTAMP	HM81_AIC	HM81_flux.estimate		
1	10/14/2022_CRC_TR_309_71	2022-10-14 10:44:29	NA	NA		
2	10/14/2022_CRC_TR_311_72	2022-10-14 10:33:24	-345.05212	0.005212169		
3	10/14/2022_CRC_TR_312_73	2022-10-14 10:26:29	-39.85766	0.009751081		
4	10/14/2022_CRC_TR_313_74	2022-10-14 11:19:44	443.86657	0.008479056		
5	10/14/2022_CRC_TR_314_75	2022-10-14 10:06:29	-578.44904	0.004443127		
6	10/14/2022_CRC_TR_315_76	2022-10-14 10:13:19	-814.74178	0.003806261		
	HM81_p.value	HM81_r.squared	HM81_RMSE	lin_AIC	lin_flux.estimate	
1	NA	NA	NA	-2886.146	0.006374834	
2	6.617145e-124	0.6275136	0.1781514	-2771.974	0.005298659	
3	8.975891e-129	0.6618584	0.2323422	-2118.675	0.007647671	
4	5.370346e-201	0.6779560	0.3174565	-3855.737	0.007162640	
5	2.769750e-234	0.6957996	0.1750786	-4199.213	0.003651375	
6	3.788822e-135	0.6410119	0.1223071	-3318.155	0.003520039	
	lin_flux.std.error	lin_int.estimate	lin_int.std.error	lin_p.value		
1	1.379411e-05	14.94248	0.001427552	0		
2	1.618581e-05	14.40415	0.001768517	0		
3	2.807453e-05	14.96527	0.002905431	0		
4	1.006980e-05	14.33155	0.001565366	0		
5	9.015396e-06	17.78667	0.001557608	0		
6	1.073807e-05	14.81559	0.001235275	0		
	lin_r.squared	lin_RMSE	poly_AIC	poly_r.squared	poly_RMSE	rob_AIC
1	0.9974873	0.01665581	-2934.846	0.9977209	0.01589215	-2885.899
2	0.9947278	0.02119477	-2853.933	0.9954659	0.01969010	-2771.853
3	0.9928020	0.03389882	-2507.631	0.9965232	0.02360348	-2106.368
4	0.9984055	0.02233745	-4315.164	0.9991002	0.01680099	-3854.836
5	0.9945554	0.02342256	-5016.651	0.9978144	0.01485684	-4198.355
6	0.9944659	0.01518574	-3346.547	0.9947567	0.01480609	-3318.138
	rob_converged	rob_flux.estimate	rob_flux.std.error	rob_RMSE		
1	TRUE	0.006381624	1.446827e-05	0.01710613		
2	TRUE	0.005293765	1.638381e-05	0.02215159		

3	TRUE	0.007726051	2.603189e-05	0.03124424
4	TRUE	0.007159559	1.003791e-05	0.02176887
5	TRUE	0.003644753	9.219273e-06	0.02465547
6	TRUE	0.003518718	1.146345e-05	0.01542092

	Timestamp_min	Timestamp_max
1	2022-10-14 10:43:00	2022-10-14 10:45:59
2	2022-10-14 10:31:50	2022-10-14 10:34:59
3	2022-10-14 10:25:00	2022-10-14 10:27:59
4	2022-10-14 11:17:30	2022-10-14 11:21:59
5	2022-10-14 10:04:00	2022-10-14 10:08:59
6	2022-10-14 10:11:40	2022-10-14 10:14:59

```
#merging fluxes with initial CO2 concentration
fluxes <- fluxes %>%
  left_join(df_CO2_in, by = "Plot")

#take a look at the fluxes we calculated with R2
ggplot(fluxes, aes(Plot, lin_flux.estimate, color = lin_r.squared)) +
  geom_point(size=4) + theme_classic() +
  geom_linerange(aes(ymin = lin_flux.estimate- lin_flux.std.error,
                    ymax = lin_flux.estimate+ lin_flux.std.error)) +
  scale_color_gradientn(colours = c("darkred", "red", "white", "lightblue", "darkblue"),
                        values = c(0, 0.5, 0.6, 0.7, 1)) +
  ylab("CO2 flux (umol/m2/s)")
```



```
#sort fluxes by R2
fluxes_good <- as.data.frame(subset(fluxes, lin_r.squared >= .90, select = Plot:TIMESTAMP_max)
fluxes_lowr2 <- as.data.frame(subset(fluxes, lin_r.squared < .90 & lin_flux.estimate < 1, select = Plot:TIMESTAMP_max)
fluxes_weird <- as.data.frame(subset(fluxes, lin_r.squared < .90, select = Plot:TIMESTAMP_max)

#find fluxes that represent the first and fourth quartile of the data
low <- quantile(fluxes$lin_flux.estimate, 0.05)
high <- quantile(fluxes$lin_flux.estimate, 0.95)

fluxes_low <- as.data.frame(subset(fluxes, lin_flux.estimate <= low, select = Plot:TIMESTAMP_max)
fluxes_high <- as.data.frame(subset(fluxes, lin_flux.estimate >= high, select = Plot:TIMESTAMP_max)

tails <- rbind(fluxes_low, fluxes_high)
```

10 check the fluxes for which I changed the start time due to lags or high values (recorded in notes of metadata)

```
#### If something is in a V shape should we take the second part to make a positive linear c
# Subset data and create relative time (seconds since flux start)
```



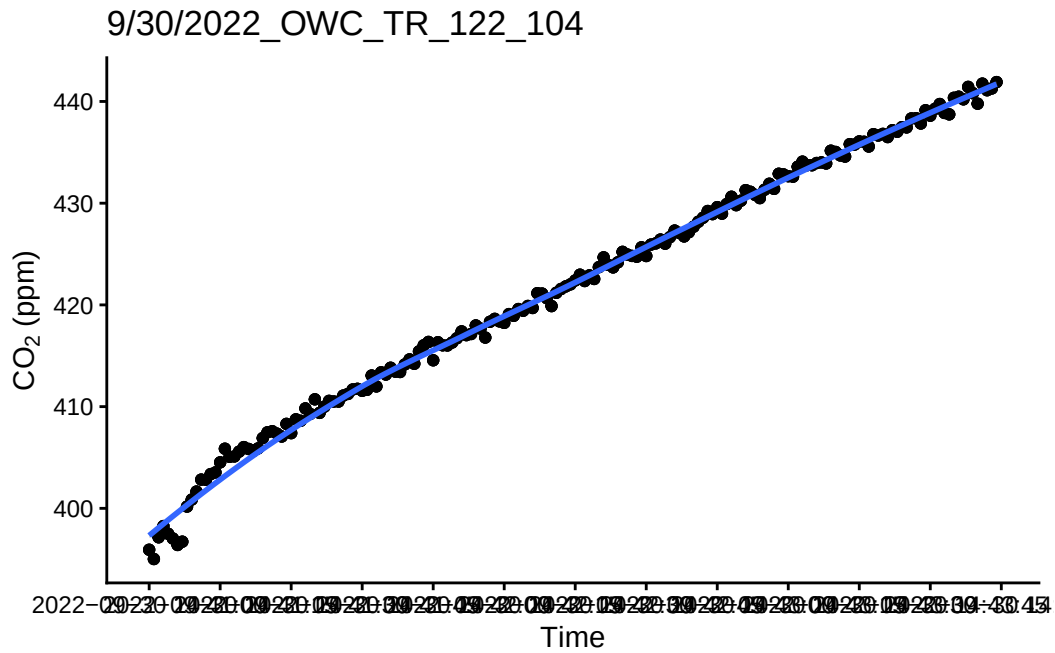
```

# r_all <- alldat %>%
#   filter(Plot %in% deadband_overrides$Plot) %>%
#   group_by(Plot) %>%
#   mutate(
#     time_sec = as.numeric(
#       difftime(TIMESTAMP, min(TIMESTAMP), units = "secs")
#     )
#   ) %>%
#   ungroup()
#
#
# # Random sample of plots for visualization
# plots_sample <- sample(unique(r_all$Plot), 6, replace = FALSE)
#
# r_plot <- r_all %>%
#   filter(Plot %in% plots_sample)
#
# # Plot
# ggplot(
#   r_plot,
#   aes(
#     x = TIMESTAMP,
#     y = co2,
#     color = time_sec >= DeadBand_used
#   )
# ) +
#   geom_point(size = 1.8) +
#   facet_wrap(~ Plot_ID, scales = "free", ncol = 2) +
#   scale_color_manual(
#     values = c("grey60", "red"),
#     labels = c("Discarded (deadband)", "Used after deadband")
#   ) +
#   theme_classic() +
#   ylab(expression(paste(CO[2], " (ppm)"))) +
#   xlab("Time")

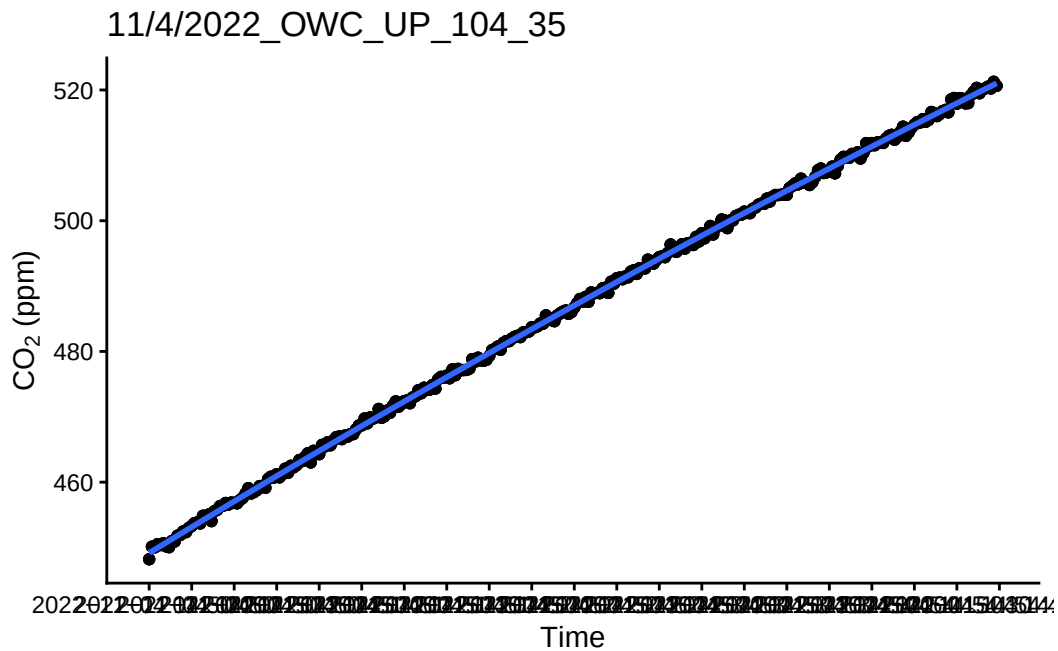
```

11 Spot check a few fluxes with $R2 > 0.9$

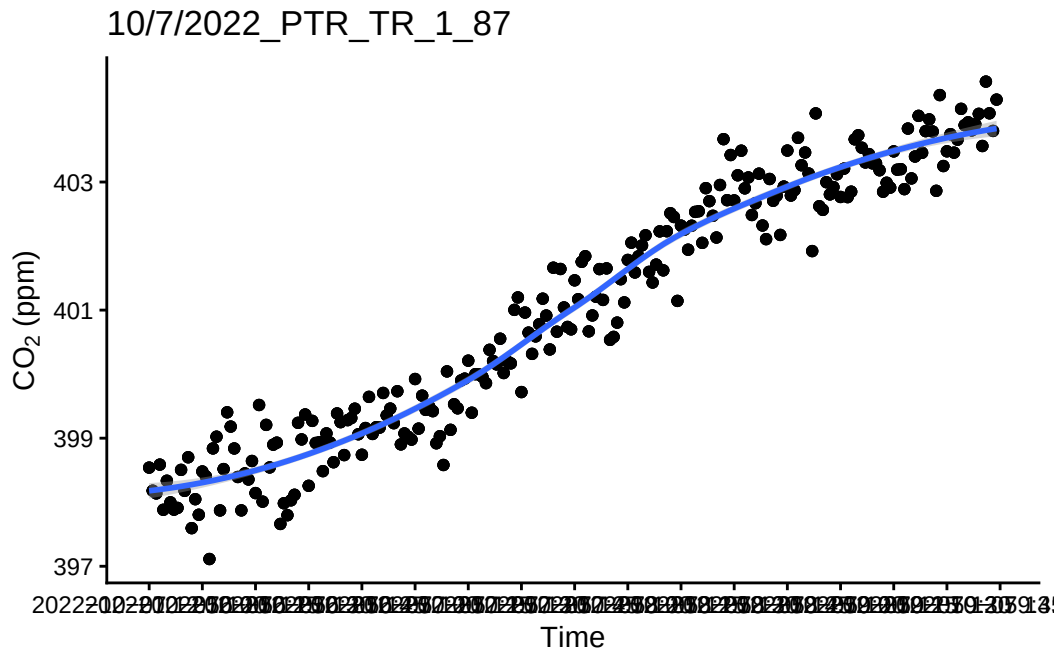
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



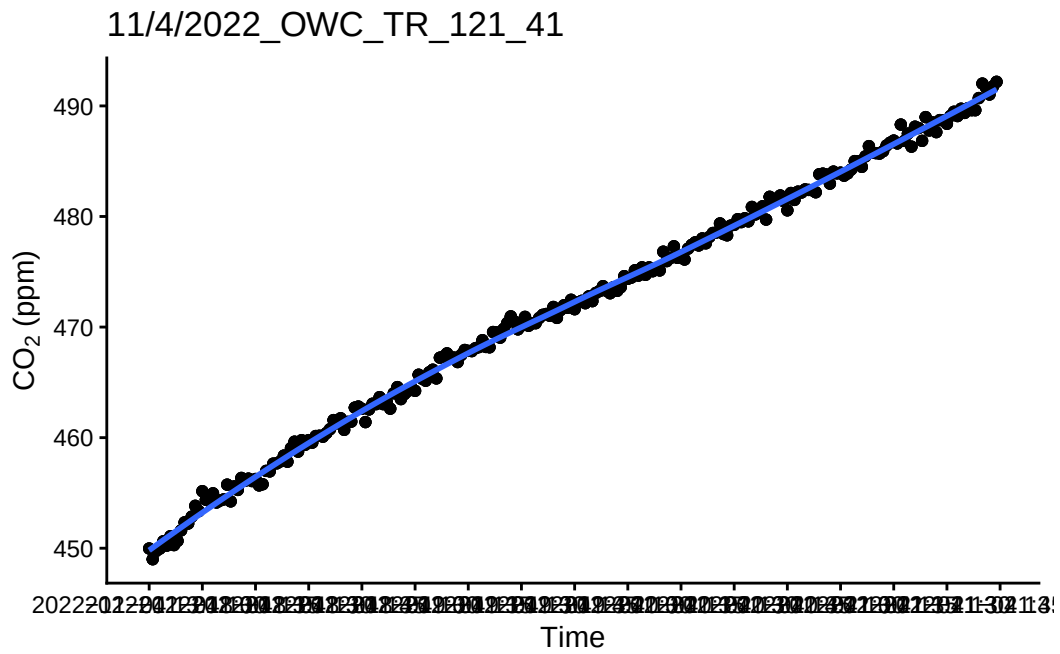
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



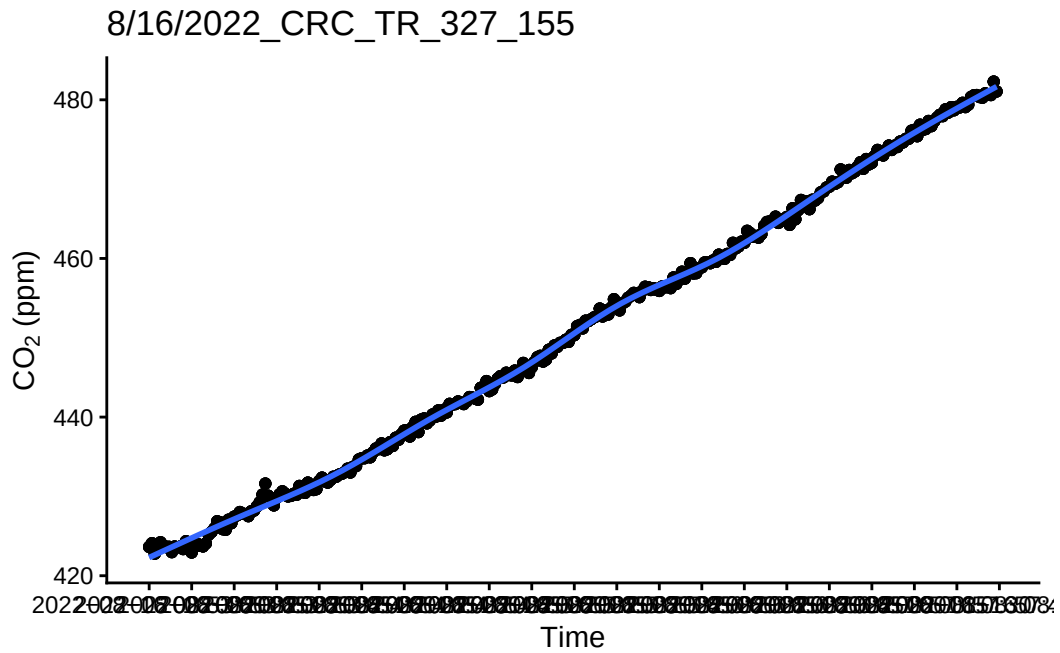
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



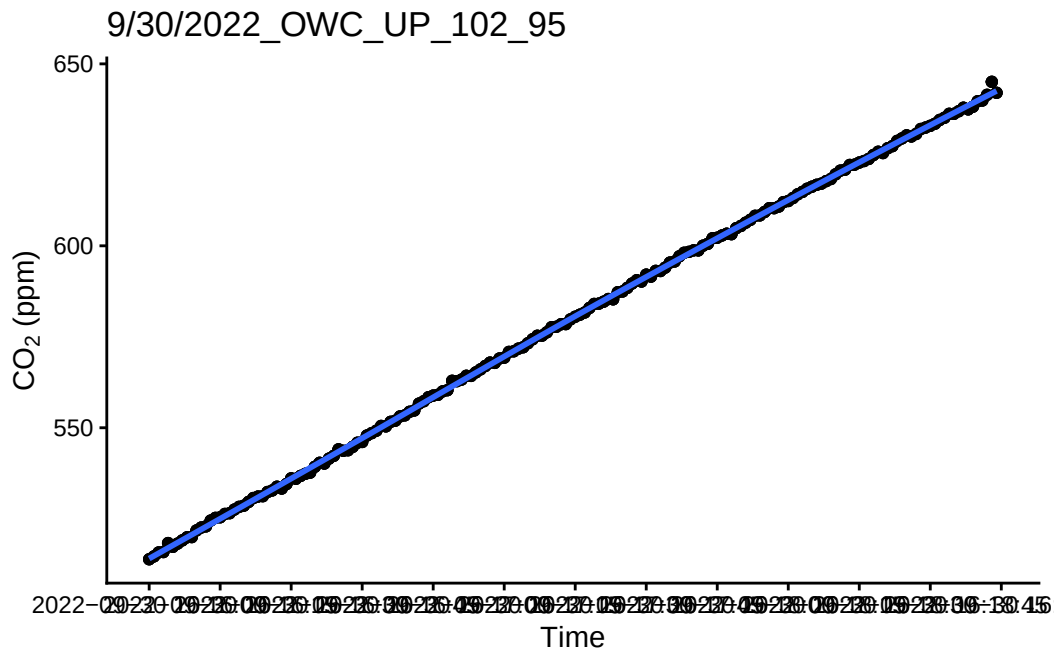
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



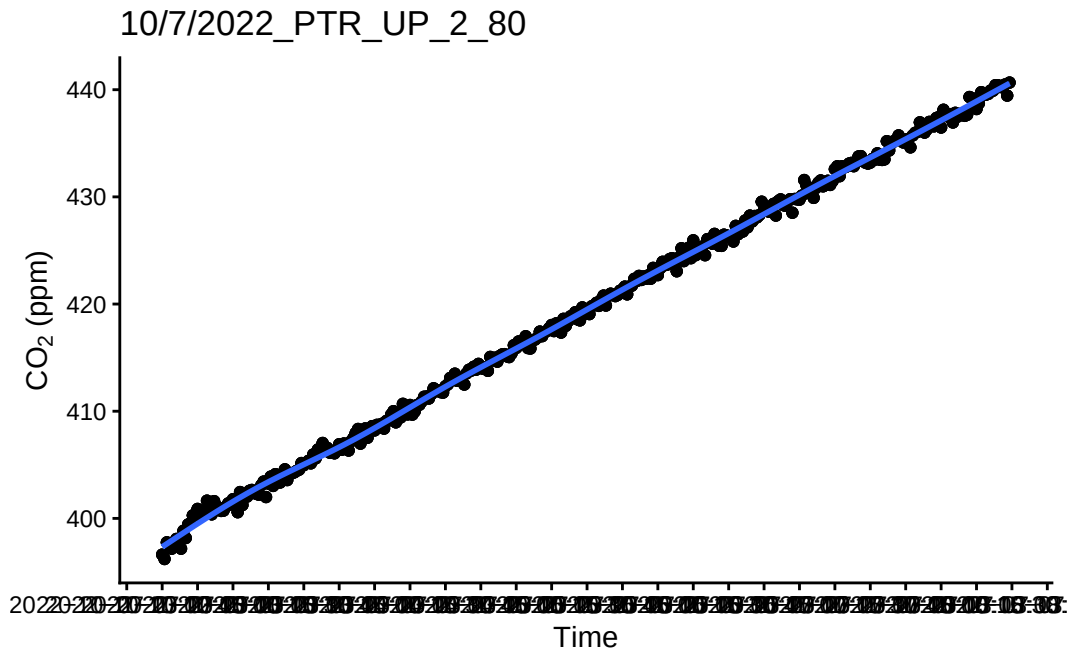
``geom_smooth()`` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'



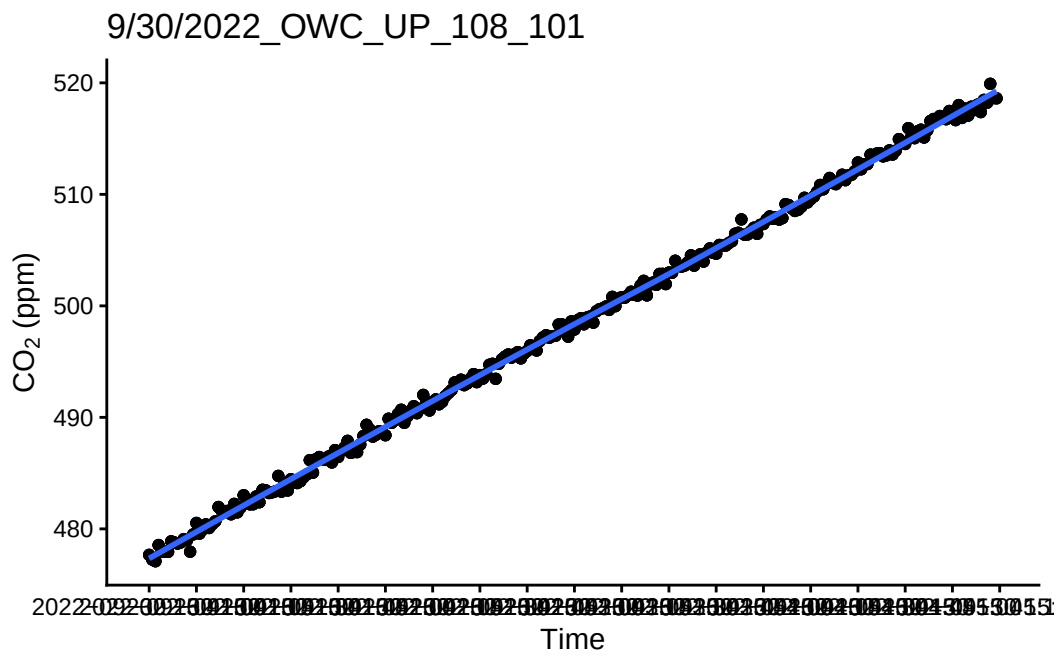
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



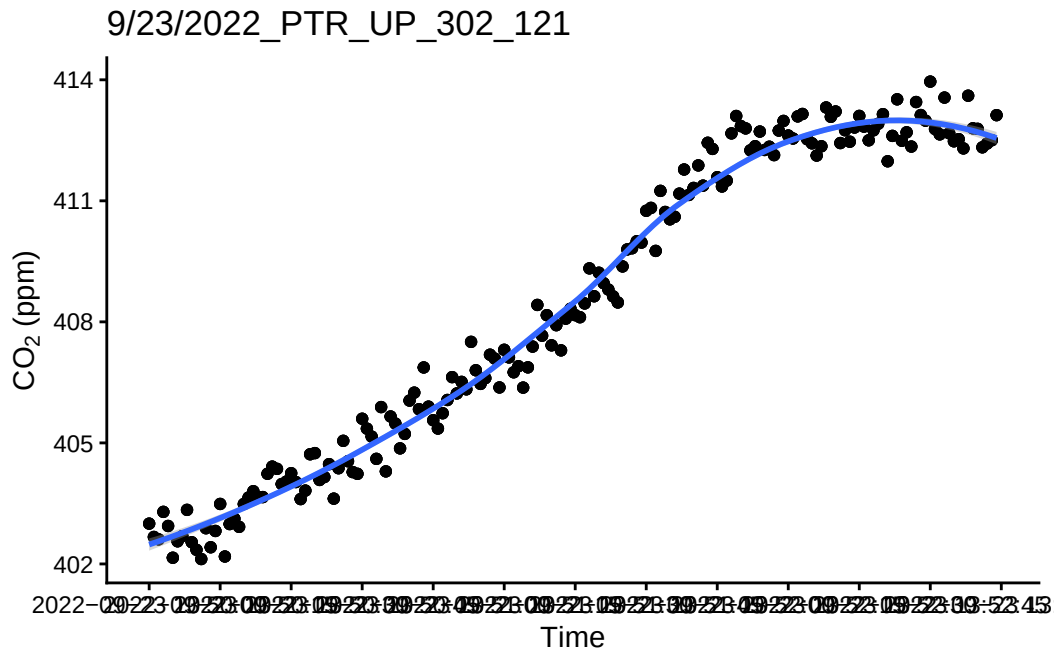
``geom_smooth()`` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

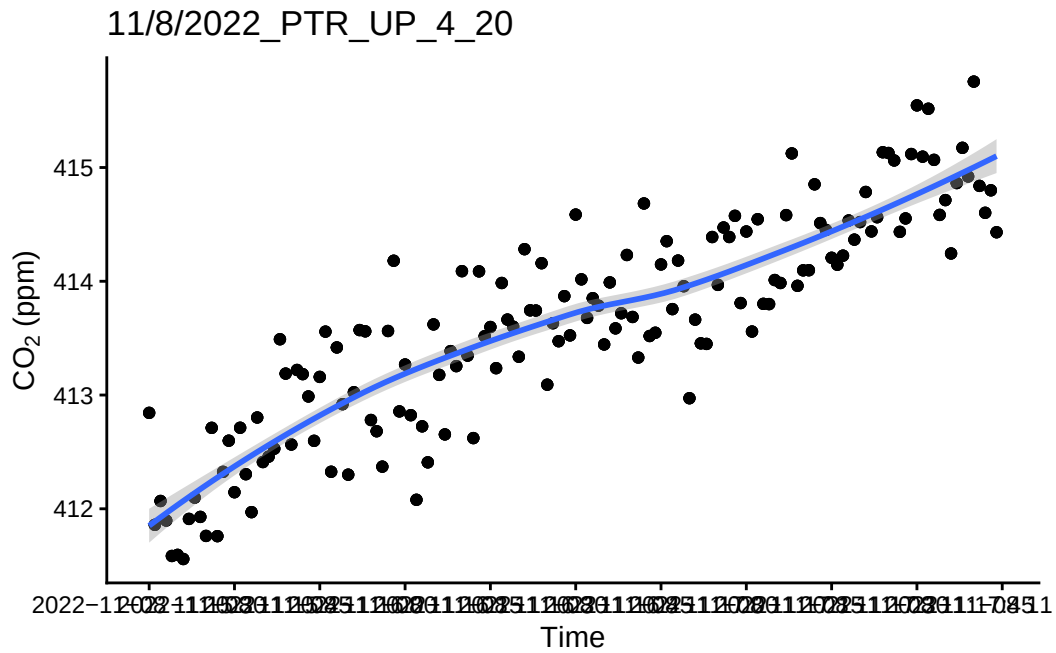


``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

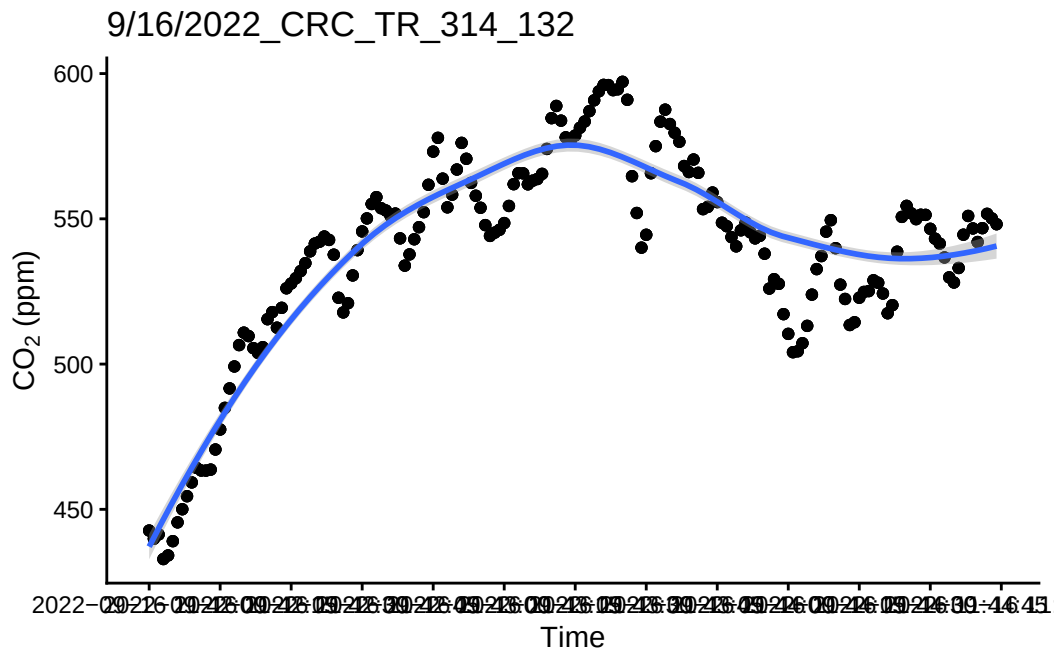


12 Take a look at the fluxes <0.9 and a flux estimate below 1

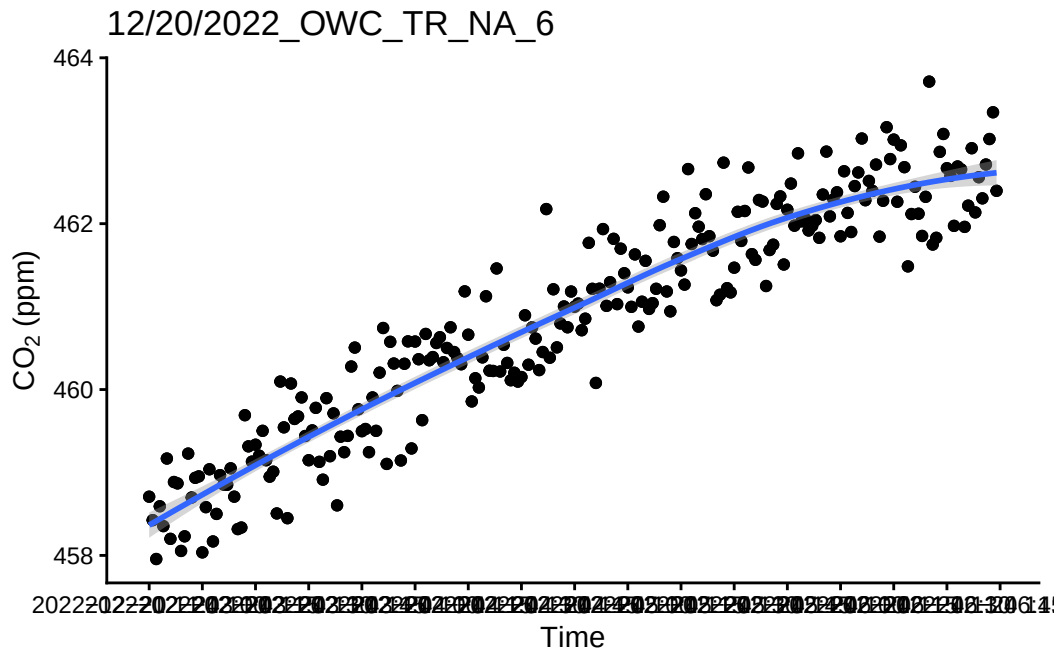
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



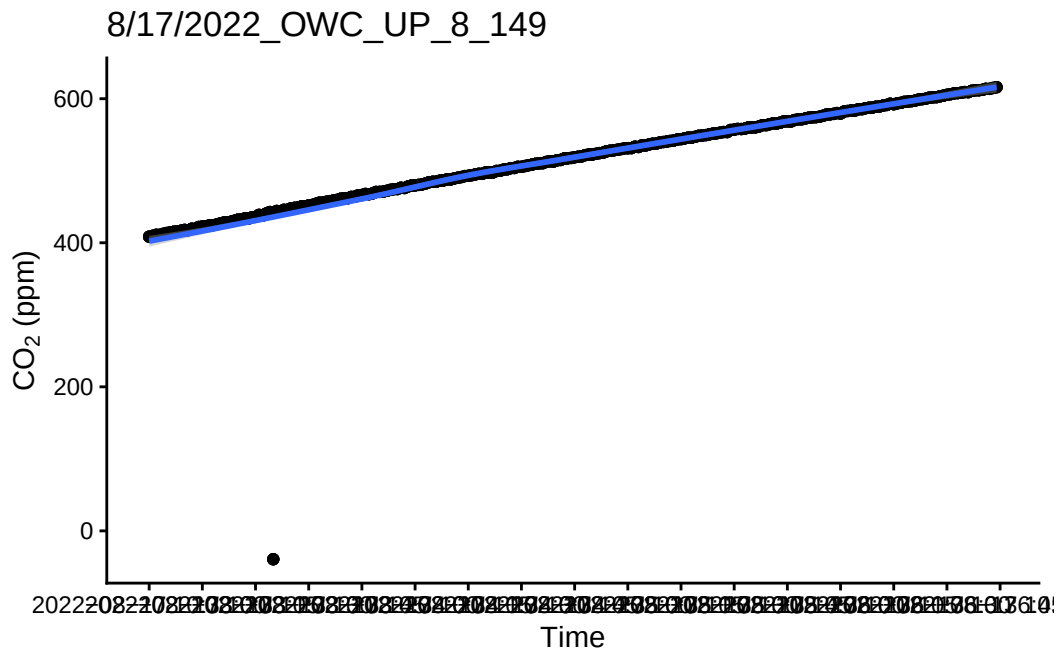
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



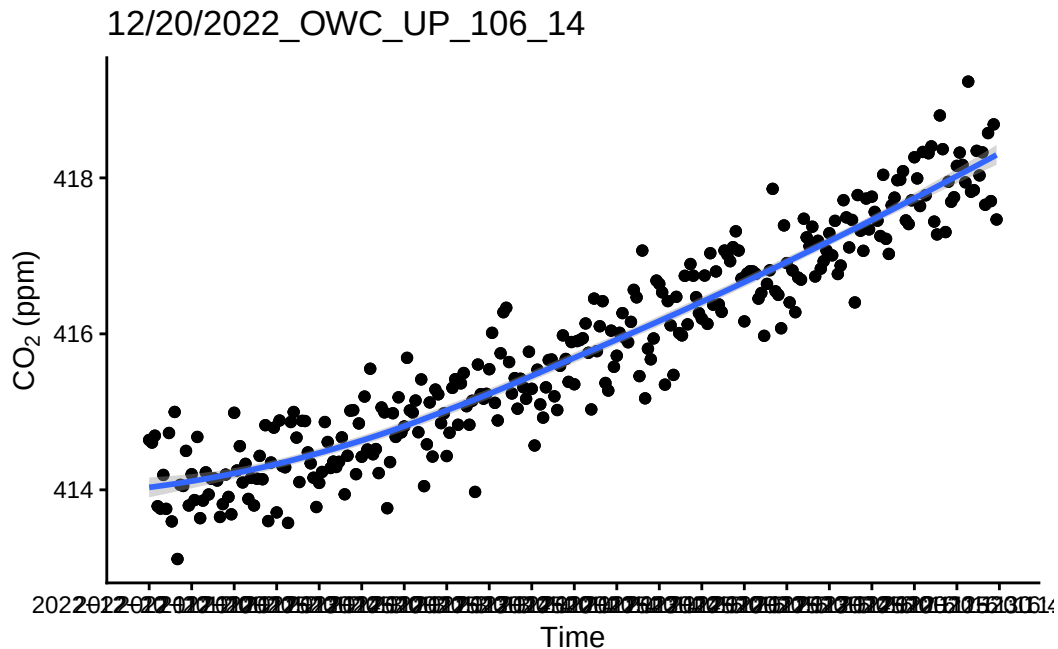
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



13 Take a look at the bad fluxes <0.9 and have a high flux estimate

14 See if we want to remove these fluxes or if we want to try and recalculate these ones?

```
### pull out fluxes with R2 < 0.9 and put them in a list
bad_plots <- unique(fluxes_weird$Plot)

# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
bad_sample <- sample(bad_plots, 5, replace=FALSE)

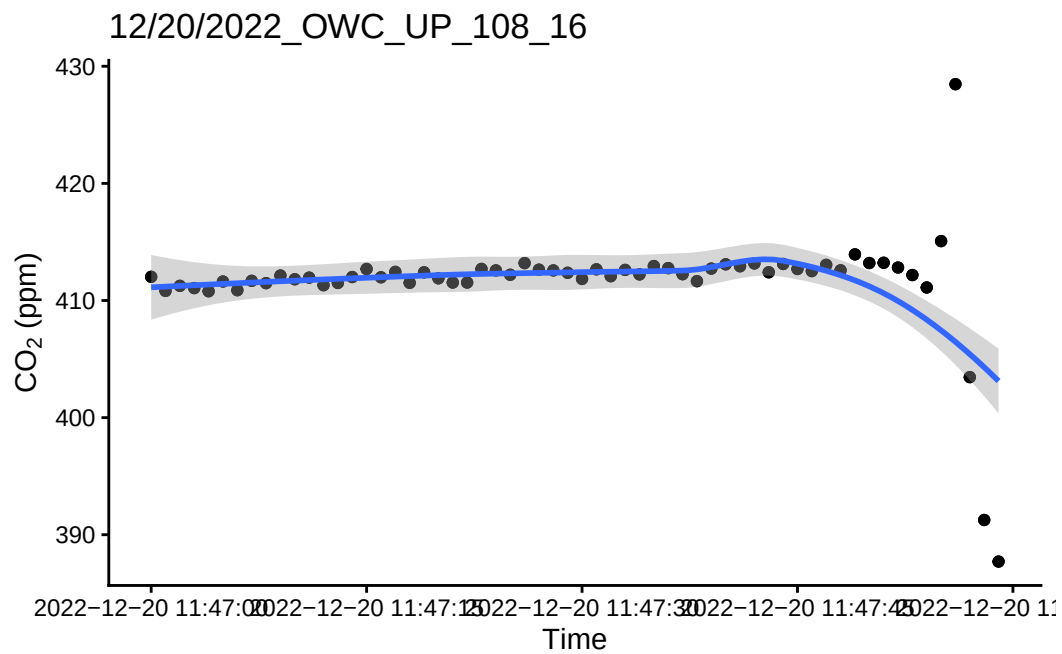
#Loop to run five random plots subset above
for(P in bad_sample){
  r1 <- subset(alldat_matched , Plot == P & !is.na(metadat_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CO[2], " (ppm)"))) +
```

```

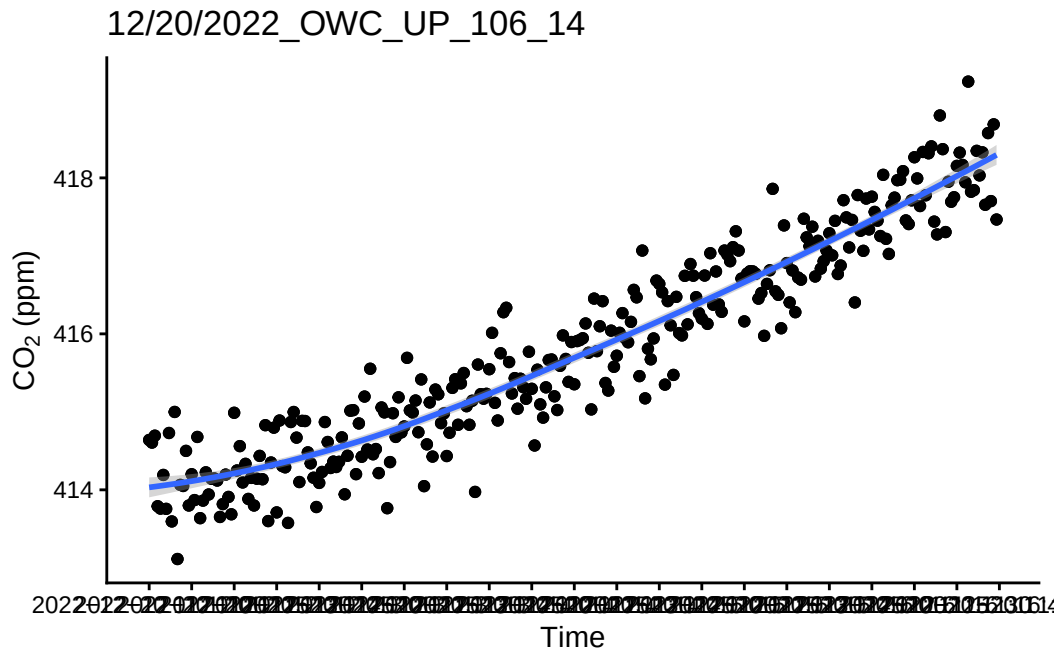
xlab("Time") + labs(title = P) +
geom_smooth()
print(p1)
}

```

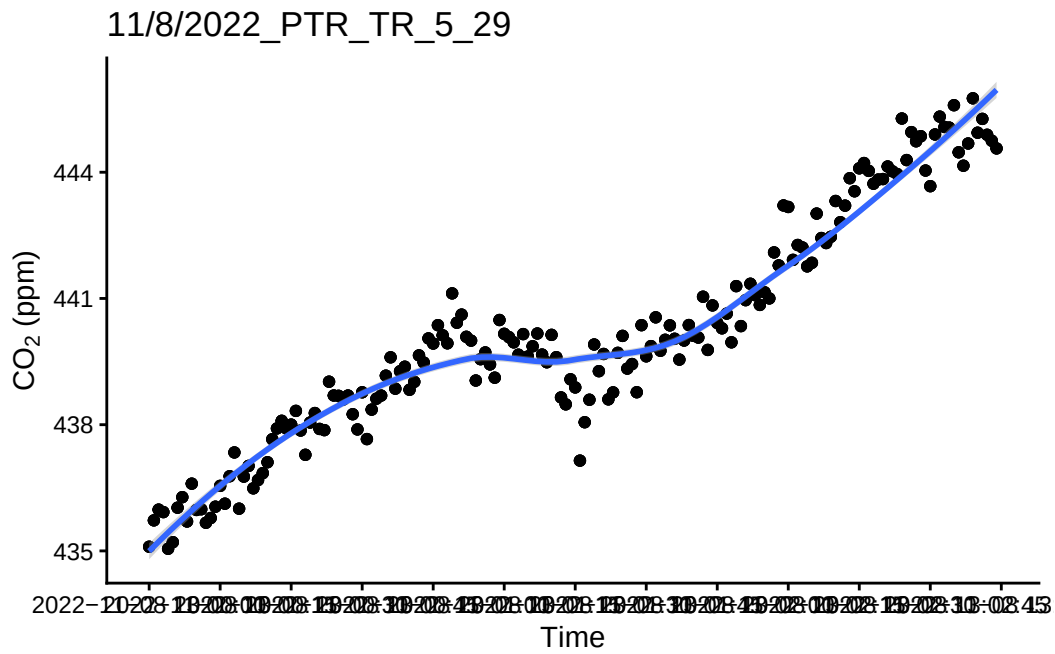
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



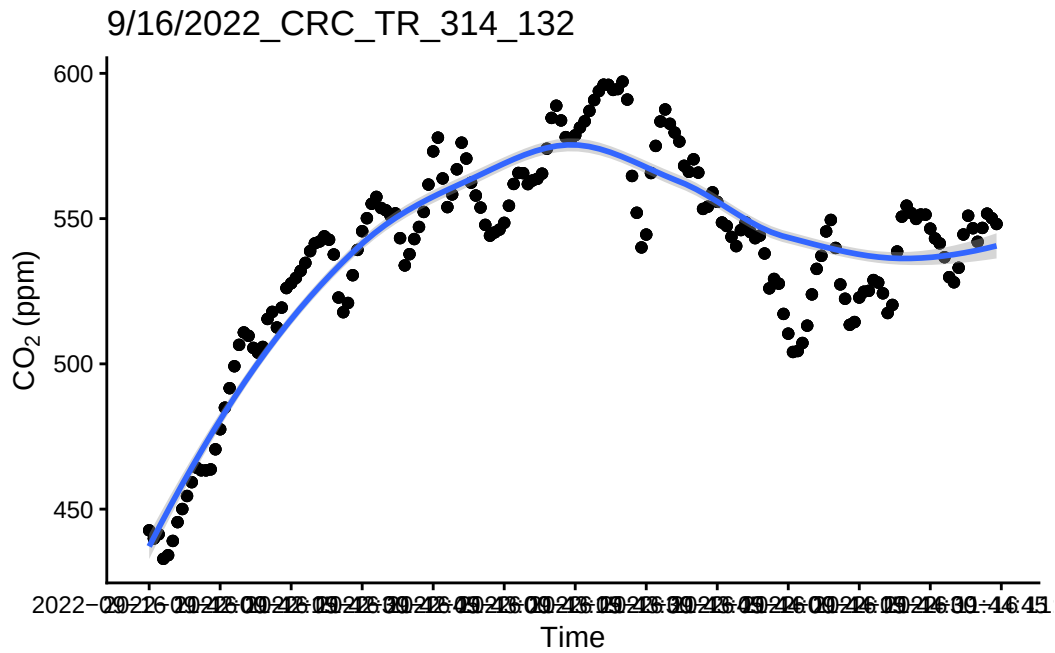
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



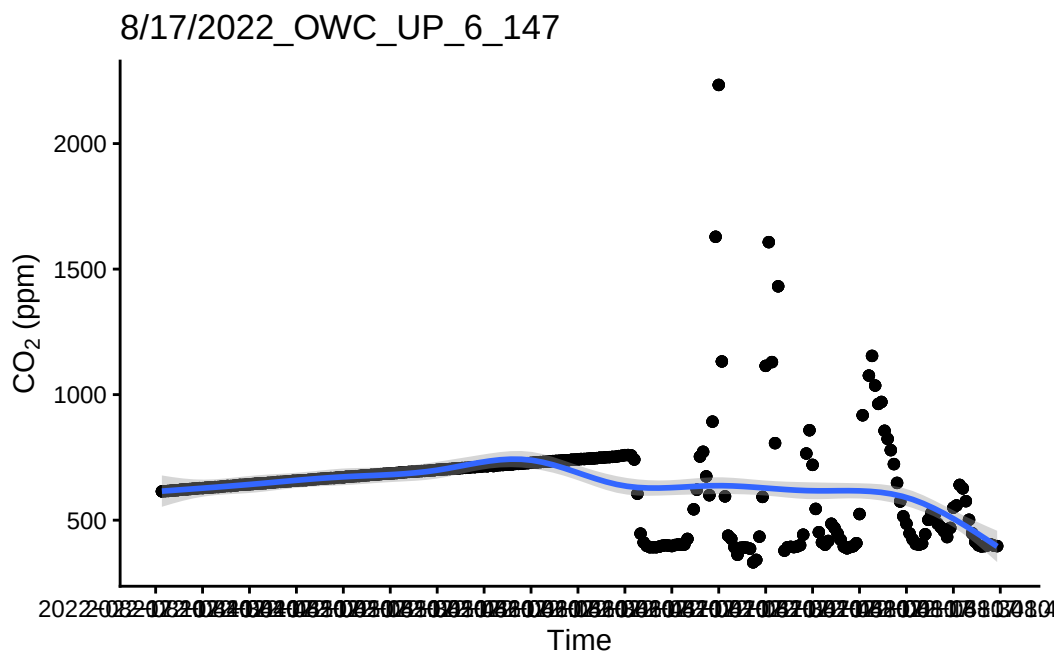
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'



```
#####
```

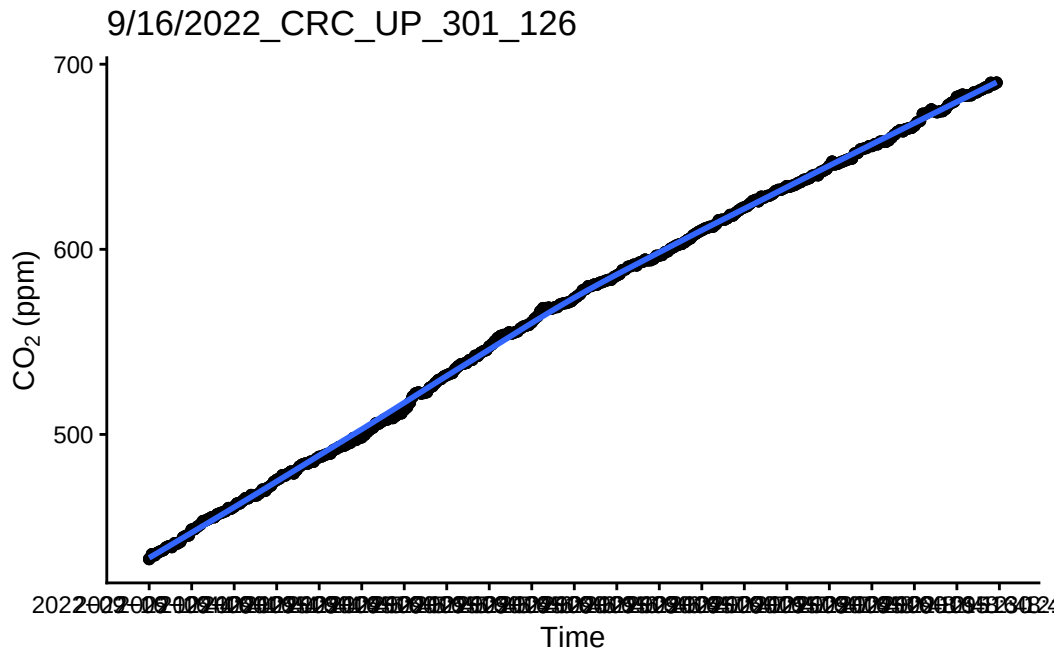
15 Look at the really high or really low ones to see if there is a timing issue or something

```
### pull out fluxes in the 5% and 95% tails to look at really high and really low
tail_plots <- unique(tails$Plot)

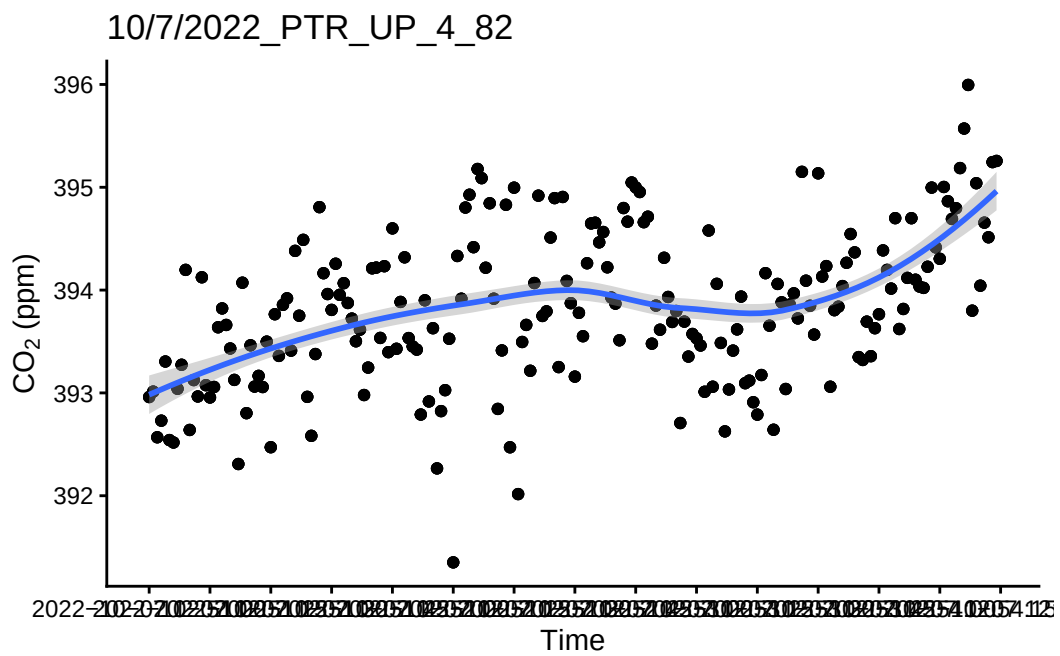
# take a random sample (FALSE so we don't get the same plot twice)
# set.seed(...) to guarantee reproducibility
tail_sample <- sample(tail_plots, 2, replace=FALSE)

#Loop to run five random plots subset above
for(P in tail_sample){
  r1 <- subset(alldat_matched , Plot == P & !is.na(metadata_row))
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +
    scale_x_datetime(breaks = "15 sec") +
    ylab(expression(paste( CO [2], " (ppm)"))) +
    xlab("Time") + labs(title = P) +
    geom_smooth()
  print(p1)
}
```

`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'

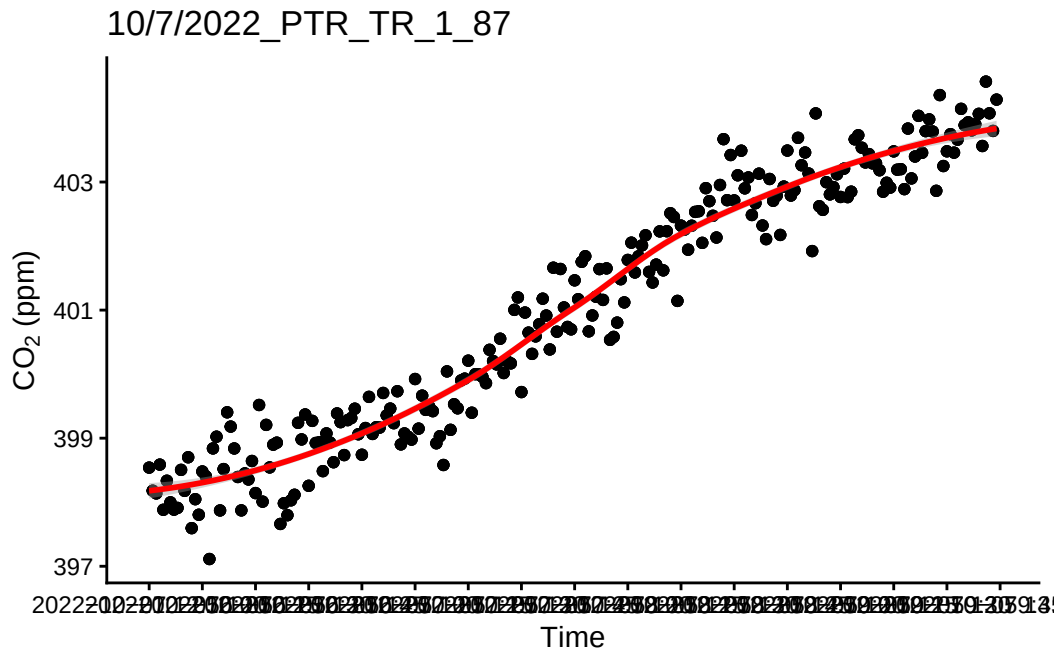


```
#would be nice to put on the plot the flux estimate and if it is a high or low or something  
#####
```

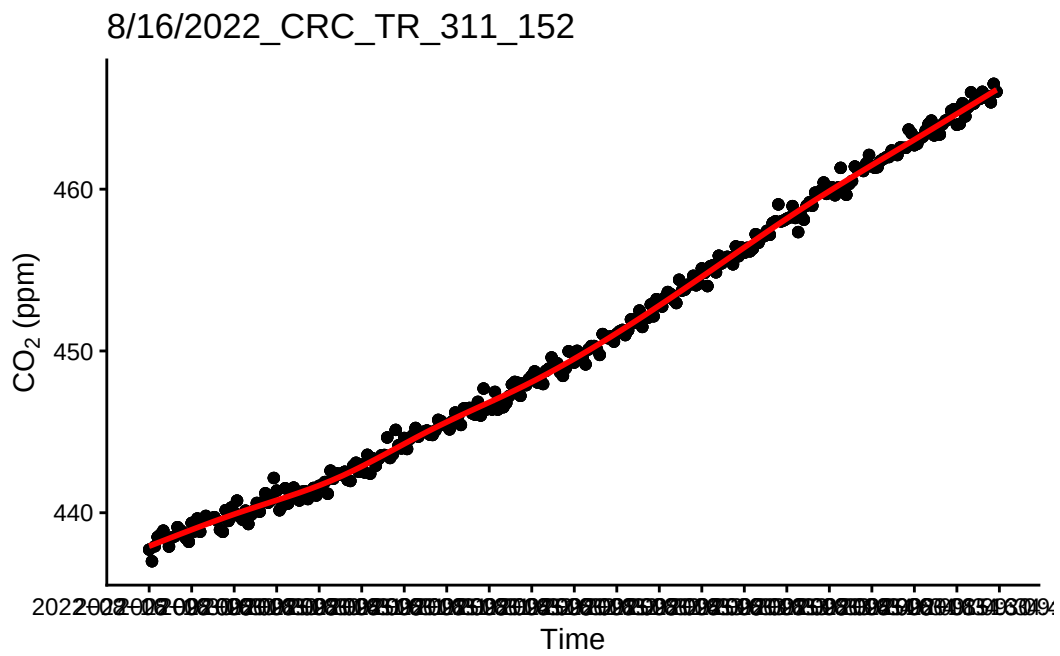
16 Check all fluxes

```
##Put all fluxes together  
fluxes_all <- as.data.frame(subset(fluxes, select = Plot:TIMESTAMP_max))  
  
all_plots <- unique(fluxes_all$Plot)  
  
# take a random sample (FALSE so we don't get the same plot twice)  
# set.seed(...) to guarantee reproducibility  
all_sample <- sample(all_plots, 5, replace=FALSE)  
  
#Loop to run five random plots subset above  
for(P in all_sample){  
  r1 <- subset(alldat_matched, Plot== P & !is.na(metadat_row))  
  p1 <- ggplot(r1, aes(TIMESTAMP, CO2)) + geom_point() + theme_classic() +  
    scale_x_datetime(breaks = "15 sec") +  
    ylab(expression(paste( CO [2], " (ppm)"))) +  
    xlab("Time") + labs(title = P) +  
    geom_smooth(color="red")  
  print(p1)  
}
```

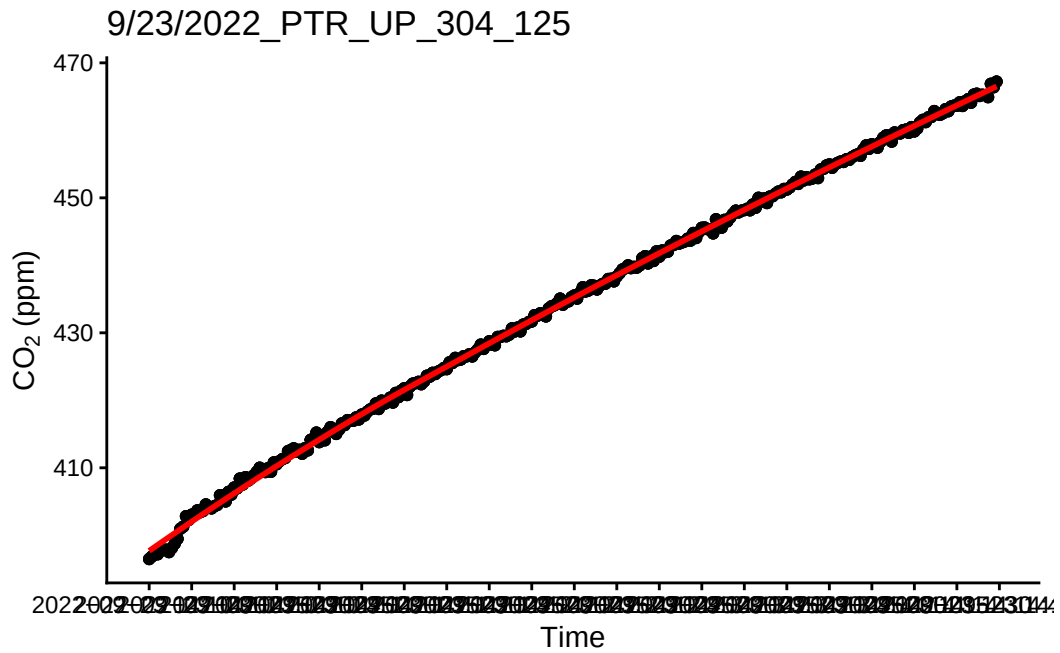
`geom\_smooth()` using method = 'loess' and formula = 'y ~ x'



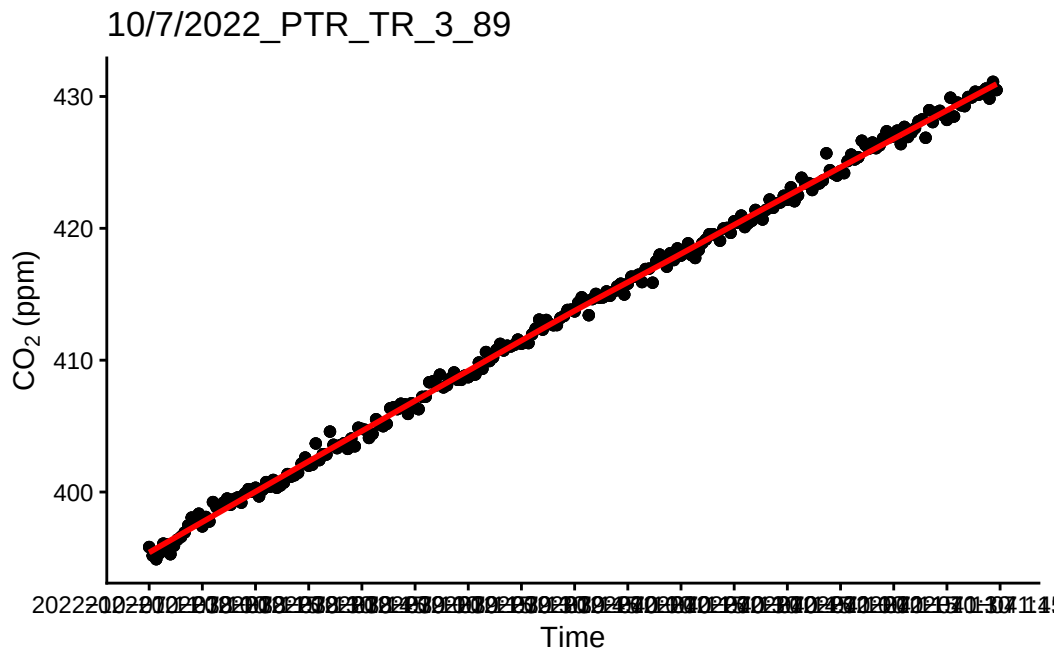
``geom_smooth()`` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'



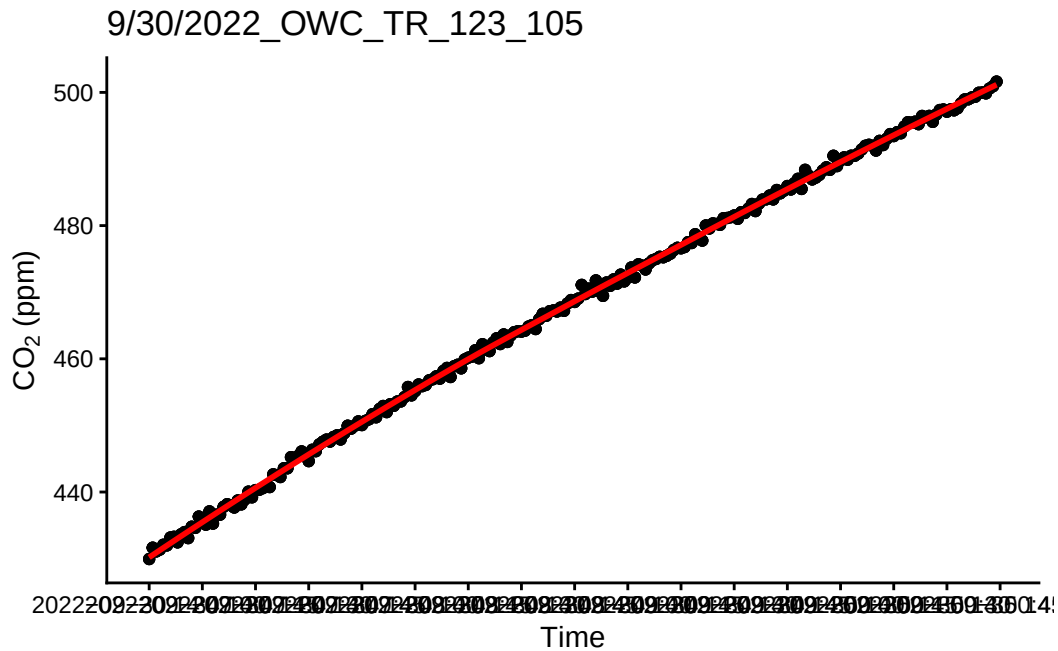
``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



``geom_smooth()`` using method = 'loess' and formula = 'y ~ x'



#####

17 Overall checks for r2

```
n_total <- nrow(fluxes_all)

r2_summary <- fluxes_all %>%
  mutate(
    lin_bad   = lin_r.squared < 0.9 | is.na(lin_r.squared),
    hm81_bad  = HM81_r.squared < 0.9 | is.na(HM81_r.squared),
    poly_bad  = poly_r.squared < 0.9 | is.na(poly_r.squared),
    fail_all_models = lin_bad & hm81_bad & poly_bad,
    pass_at_least_one = !(fail_all_models),
    neg_CO2 = lin_flux.estimate < 0 | is.na(lin_flux.estimate)
  ) %>%
  summarise(
    total_obs = n(),
    n_lin_r2_below_0.9 = sum(lin_bad, na.rm = TRUE),
    pct_lin_r2_below_0.9 = 100 * n_lin_r2_below_0.9 / total_obs,
    n_fail_all_models = sum(fail_all_models, na.rm = TRUE),
```

```

    pct_fail_all_models = 100 * n_fail_all_models / total_obs,
    n_pass_at_least_one = sum(pass_at_least_one, na.rm = TRUE),
    pct_pass_at_least_one = 100 * n_pass_at_least_one / total_obs,
    n_neg_CO2 = sum(neg_CO2, na.rm = TRUE),
    pct_neg_CO2 = 100 * n_neg_CO2 / total_obs
  )
r2_summary

```

```

total_obs n_lin_r2_below_0.9 pct_lin_r2_below_0.9 n_fail_all_models
1 117363 10126 8.627932 7906
pct_fail_all_models n_pass_at_least_one pct_pass_at_least_one n_neg_CO2
1 6.736365 109457 93.26364 3946
pct_neg_CO2
1 3.362218

```

18 Remove any fluxes that do not fit criteria

```

# Create a new dataframe by removing rows that were present in fluxes_weird ???
#allflux_filtered <- allflux[!(allflux$Plot %in% fluxes_weird$Plo), ] # not all of them are l

#Any others that are not in the weird category that need to be removed:
#also removing ones that are crazy off but were not in the fluxes_weird dataframe

allflux_filtered <- fluxes %>%
  filter(!Plot %in% c("2022_08_17_OWC_UP_7_148", "2022_11_24_OWC_TR_123_43"))

# # View the result
head(allflux_filtered)

```

```

          Plot          TIMESTAMP HM81_AIC HM81_flux.estimate
1 10/14/2022_CRC_TR_309_71 2022-10-14 10:44:29          NA          NA
2 10/14/2022_CRC_TR_309_71 2022-10-14 10:44:29          NA          NA
3 10/14/2022_CRC_TR_309_71 2022-10-14 10:44:29          NA          NA
4 10/14/2022_CRC_TR_309_71 2022-10-14 10:44:29          NA          NA
5 10/14/2022_CRC_TR_309_71 2022-10-14 10:44:29          NA          NA
6 10/14/2022_CRC_TR_309_71 2022-10-14 10:44:29          NA          NA
  HM81_p.value HM81_r.squared HM81_RMSE  lin_AIC lin_flux.estimate

```

1	NA	NA	NA	-2886.146	0.006374834
2	NA	NA	NA	-2886.146	0.006374834
3	NA	NA	NA	-2886.146	0.006374834
4	NA	NA	NA	-2886.146	0.006374834
5	NA	NA	NA	-2886.146	0.006374834
6	NA	NA	NA	-2886.146	0.006374834
lin_flux.std.error lin_int.estimate lin_int.std.error lin_p.value					
1	1.379411e-05	14.94248	0.001427552		0
2	1.379411e-05	14.94248	0.001427552		0
3	1.379411e-05	14.94248	0.001427552		0
4	1.379411e-05	14.94248	0.001427552		0
5	1.379411e-05	14.94248	0.001427552		0
6	1.379411e-05	14.94248	0.001427552		0
lin_r.squared lin_RMSE poly_AIC poly_r.squared poly_RMSE rob_AIC					
1	0.9974873	0.01665581	-2934.846	0.9977209	0.01589215 -2885.899
2	0.9974873	0.01665581	-2934.846	0.9977209	0.01589215 -2885.899
3	0.9974873	0.01665581	-2934.846	0.9977209	0.01589215 -2885.899
4	0.9974873	0.01665581	-2934.846	0.9977209	0.01589215 -2885.899
5	0.9974873	0.01665581	-2934.846	0.9977209	0.01589215 -2885.899
6	0.9974873	0.01665581	-2934.846	0.9977209	0.01589215 -2885.899
rob_converged rob_flux.estimate rob_flux.std.error rob_RMSE					
1	TRUE	0.006381624	1.446827e-05	0.01710613	
2	TRUE	0.006381624	1.446827e-05	0.01710613	
3	TRUE	0.006381624	1.446827e-05	0.01710613	
4	TRUE	0.006381624	1.446827e-05	0.01710613	
5	TRUE	0.006381624	1.446827e-05	0.01710613	
6	TRUE	0.006381624	1.446827e-05	0.01710613	
TIMESTAMP_min TIMESTAMP_max CO2_initial					
1	2022-10-14 10:43:00	2022-10-14 10:45:59	435.6188		
2	2022-10-14 10:43:00	2022-10-14 10:45:59	435.6188		
3	2022-10-14 10:43:00	2022-10-14 10:45:59	435.6188		
4	2022-10-14 10:43:00	2022-10-14 10:45:59	435.6188		
5	2022-10-14 10:43:00	2022-10-14 10:45:59	435.6188		
6	2022-10-14 10:43:00	2022-10-14 10:45:59	435.6188		

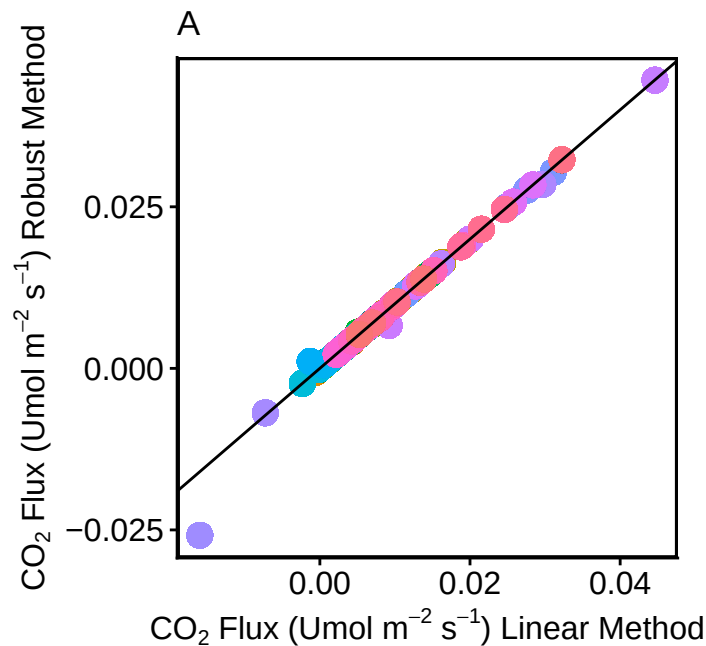
19 Do some visualization of QAQC plots to assess fluxes

```
p1 <- ggplot(allflux_filtered, aes(lin_flux.estimate, rob_flux.estimate, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
```

```

labs(title = "A") +
ylab(expression(paste( CO [2], " Flux (Umol m-2* " s-1*) Robust Method"))) +
xlab(expression(paste( CO [2], " Flux (Umol m-2* " s-1*) Linear Method"))) +
guides(color = guide_legend(title = "Plot")) +
theme(legend.position="none") +
theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
      axis.title.y = element_text(size=12), legend.text=element_text(size=12),
      panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
      aspect.ratio = 1)
p1

```



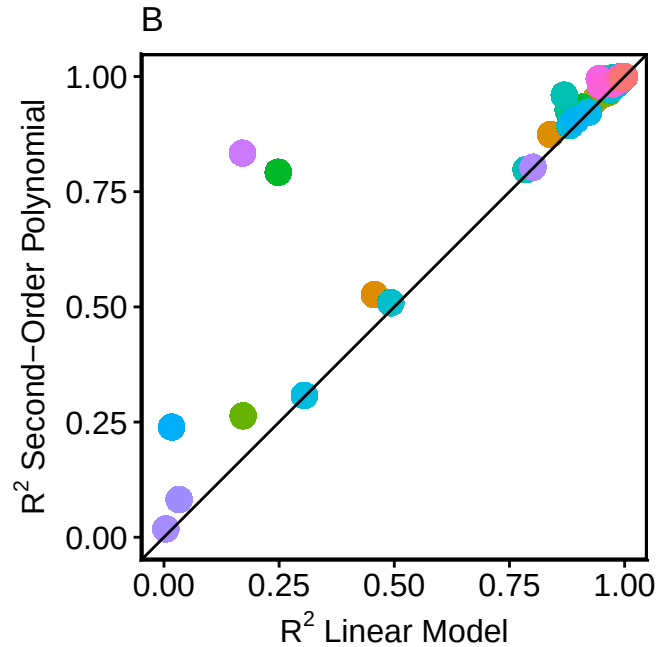
```

p2 <- ggplot(allflux_filtered, aes(lin_r.squared, poly_r.squared, color = Plot)) +
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  xlim(0,1) + ylim(0,1) +
  labs(title = "B") +
  ylab(expression(paste("R"2* " Second-Order Polynomial")))) +
  xlab(expression(paste("R"2* " Linear Model")))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="none") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),

```

```
panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
aspect.ratio = 1)
```

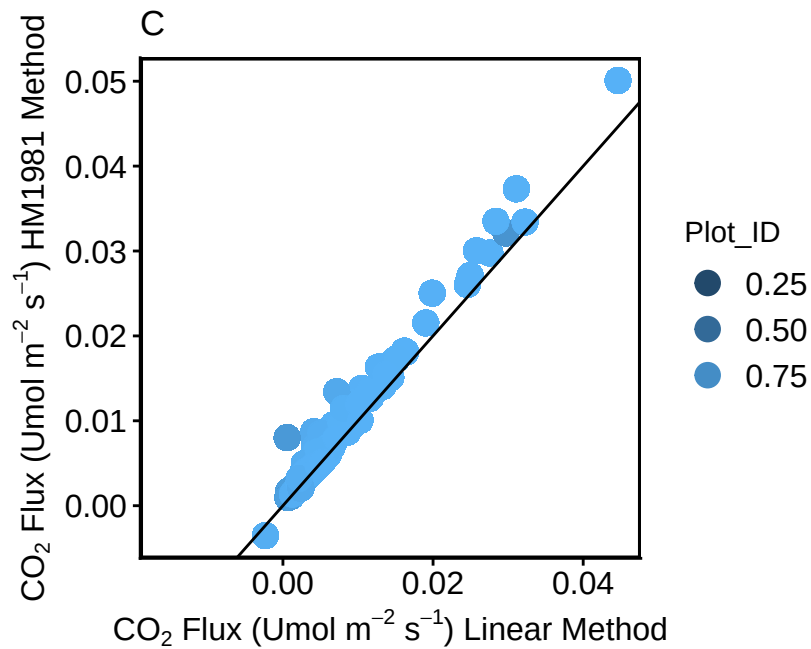
p2



```
p3 <- ggplot(allflux_filtered, aes(lin_flux.estimate, HM81_flux.estimate, color = lin_r.squa
  geom_point(size=4) + geom_abline() +
  theme_classic() +
  #xlim(0,50) + #ylim(0,50) +
  labs(title = "C") +
  ylab(expression(paste( CO [2], " Flux (Umol m"^-2* " s"^-1*)" HM1981 Method")))) +
  xlab(expression(paste( CO [2], " Flux (Umol m"^-2* " s"^-1*)" Linear Method")))) +
  guides(color = guide_legend(title = "Plot_ID")) +
  theme(legend.position="right") +
  theme(axis.title.x = element_text(size=12), axis.text = element_text(size=12),
        axis.title.y = element_text(size=12), legend.text=element_text(size=12),
        panel.border = element_rect(colour = "black", fill=NA, linewidth =1),
        aspect.ratio = 1)
```

p3

Warning: Removed 27304 rows containing missing values or values outside the scale range (``geom_point()``).



```
#ggarrange(p1, p2, p3, ncol=3, nrow=1, common.legend = TRUE, legend="none")
```

20 Read out final dataframe and export for use elsewhere

```
allflux_filtered_renamed <- fluxes %>%
  mutate(
    Date = mdy(str_extract(Plot, "[0-9]+/[0-9]+/[0-9]{4}")),
    Month = month.abb[month(Date)],
    Site = str_extract(Plot, "(PTR|OWC|CRC)"),
    Zone = str_extract(Plot, "_TR_|_UP_") %>%
      str_remove_all("_"),
    Replicate = as.integer(
      str_extract(Plot, "(?<=_ (TR|UP)_)[0-9]+")
    ),
    Plot_ID = as.integer(
      str_extract(Plot, "[0-9]+$") ) %>%

# Optional: rebuild a clean Plot label
mutate(
  Plot_clean = paste(
```

```

        format(Date, "%Y-%m-%d"),
        Site,
        Zone,
        Replicate,
        Plot_ID,
        sep = "_"
    )
) %>%

select(
  Plot_clean,
  Date,
  Month,
  Site,
  Zone,
  Replicate,
  Plot_ID,
  everything(),
  -Plot
)

#export as a csv to the folder that you want
write.csv(allflux_filtered_renamed, file="Processed Data/COMPASS_Synoptic_LE_TreeStem_Fluxes.csv", as.is=T)
# #Change file name

```

21